

Lecture 1



香港城市大學
City University of Hong Kong
Innovating into the Future

Introduction: Perception & Generation

CS5494 Topics in Generative AI

Dr. Jianyuan Guo

Fall 2026, Dept. CS, CityU

Overview



- Perception Model
 - Representation Learning
 - Deep neural networks
- Generation Model
 - Learning to represent probability distributions

Acknowledgement

This presentation is based on materials from:

- [6.S978: Deep Generative Models] by [Kaiming He/MIT], [2024]
- [CS236 : Deep Generative Models] by [Stefano Ermon/Stanford], [2023]

The slides have been adapted and extended with additional content, examples, and illustrations for the purpose of this lecture.

The Generative AI Era

Chatbot and natural language conversation

Gemini

2.5 Flash ▾

What are generative models?

◆ Generative models are a type of machine learning model designed to create new data that is similar to the data they were trained on. Unlike discriminative models, which learn to classify or predict outcomes based on given inputs (e.g., classifying an image as a "cat" or "dog"), generative models learn the underlying patterns and distribution of the data itself. This allows them to "generate" new, unique content, such as text, images, audio, and more. Think of it like this: a discriminative model can tell you if a picture is a cat, but a generative model can draw a new, original picture of a cat.    

How Generative Models Work

Generative models learn to capture the joint probability distribution of a dataset. This means they learn how different features of the data relate to each other. For instance, in a model trained on images of human faces, it learns the relationships between the positions of eyes, the shape of a nose, and the presence of a mouth. After learning these complex relationships, the model can generate new data that follows the same statistical patterns.   

The training process for these models often involves unsupervised learning, where the model is given a large dataset without labels and must figure out the data's internal logic on its own.

Ask Gemini

+ Tools



ChatGPT ▾

What are generative models?

Generative models are a class of machine learning models that learn the underlying distribution of data so they can generate new samples that resemble the training data.

In simple terms: instead of just learning to classify or predict (like *discriminative models*), generative models try to **model how the data was created**, allowing them to generate new, realistic-looking data points.

Key Ideas:

- They capture **joint probability distributions** $P(x, y)$ or just the data distribution $P(x)$.
- They can generate synthetic data, such as images, text, audio, or videos.
- They are often trained on large datasets and learn to mimic the patterns in them.

Examples of Generative Models:

1. **Gaussian Mixture Models (GMMs)** – early statistical generative models.
2. **Hidden Markov Models (HMMs)** – used for sequences like speech.
3. **Naive Bayes** (in a sense, since it models $P(x|y)$ and $P(y)$).
4. **Modern Deep Generative Models:**
 - **Variational Autoencoders (VAEs)** – generate data by sampling from a latent space.
 - **Generative Adversarial Networks (GANs)** –  in a generator and discriminator in a minimax game.
 - **Diffusion Models** – iteratively denoise random noise into realistic samples (e.g., Stable Diffusion, DALL-E).

+ Ask anything



The Generative AI Era

Text-to-image generation



Generated by Stable Diffusion 3 Medium. Prompt: teddy bear teaching a course, with "generative models" written on blackboard

The Generative AI Era

Text-to-video generation



The Generative AI Era

Text-to-video generation



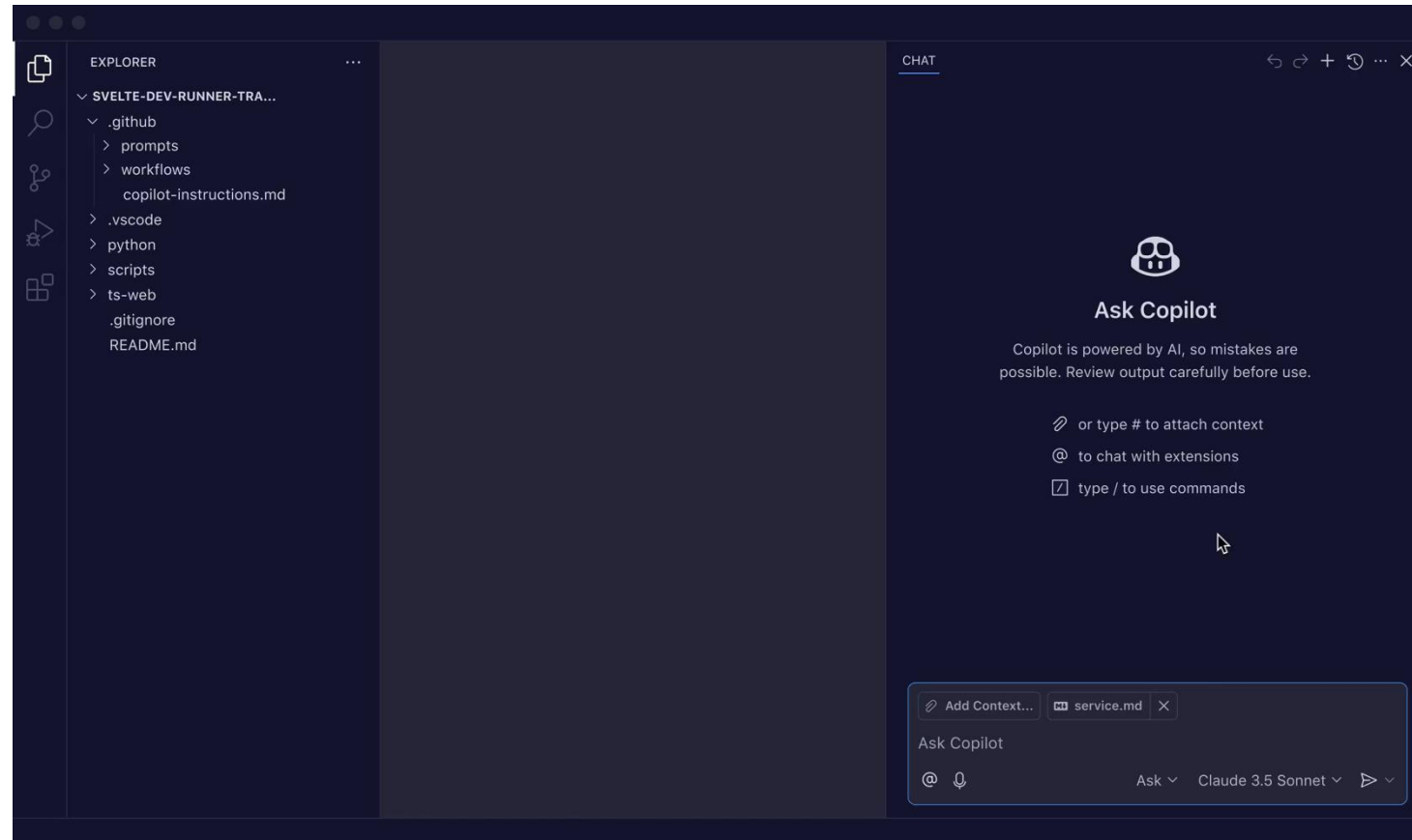
Generated by Veo3.



Generated by KlingAI.

The Generative AI Era

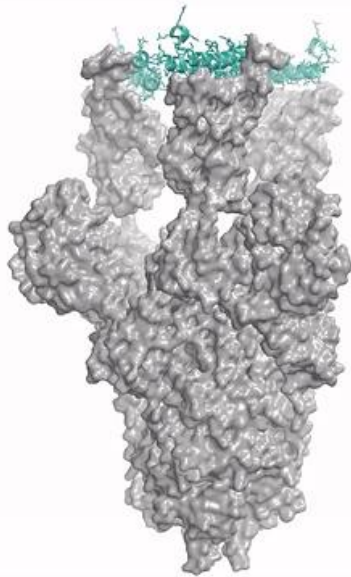
AI assistant for code generation



<https://github.com/features/copilot>

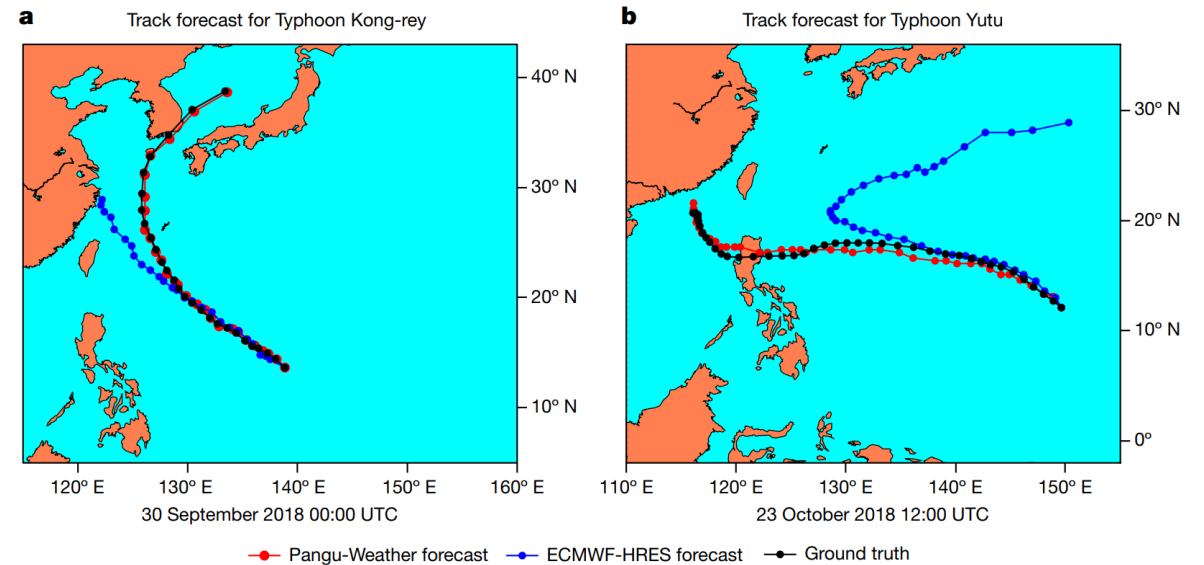
The Generative AI Era

Protein design and generation



Watson, et al. "De novo design of protein structure and function with Rfdiffusion." Nature 2023

Weather forecasting



Bi, et al. "Accurate medium-range global weather forecasting with 3D neural networks." Nature 2023

Generative Models before the GenAI Era

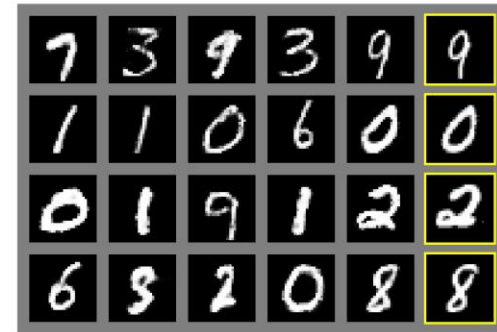
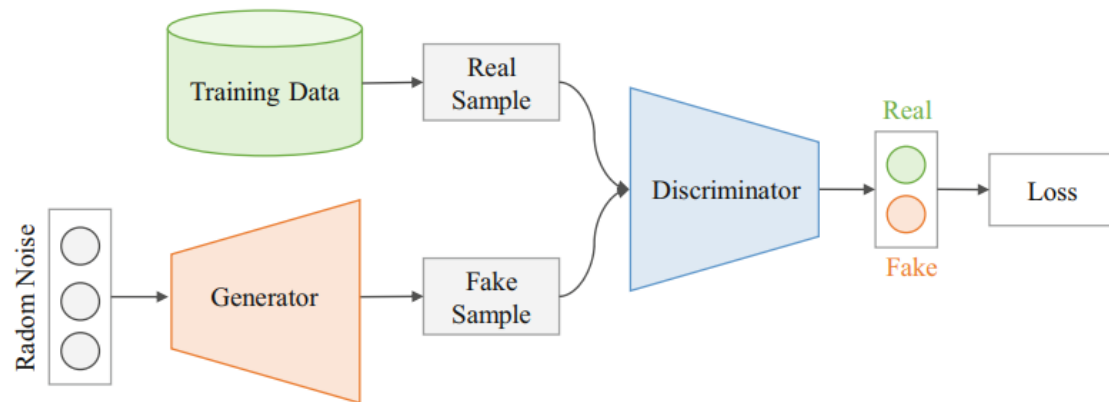
2009, PatchMatch: Photoshop's Content-aware Fill



Barnes, Connelly, et al. "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing", SIGGRAPH 2009

The Beginning of the GenAI Era

2014 Generative Adversarial Nets (GANs)



a)



b)



c)



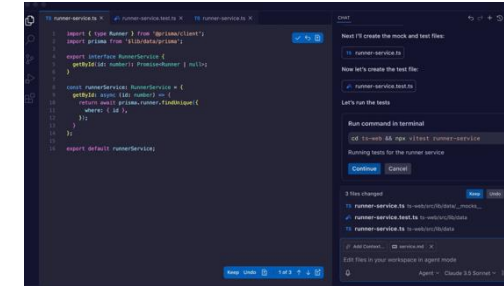
d)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

What are Generative Models?

What do these scenarios have in common?

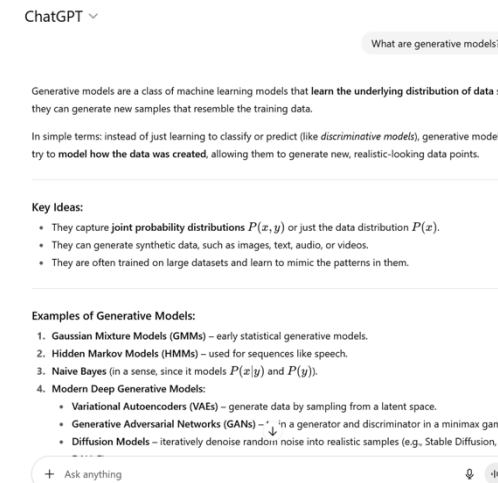
- There are **multiple** or infinite predictions to one input.
- Some predictions are more “**plausible**” than some others.
- Training data may contain **no exact solution**.
- Predictions may be **more complex, more informative**, and higher-dimensional than input.



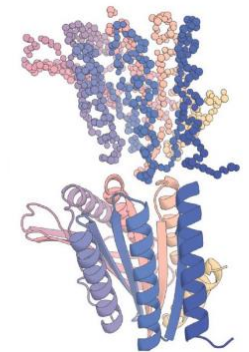
Code Generation



Image Generation



Text Generation



Protein Generation

Perception (Discriminative) vs. Generative models

discriminative

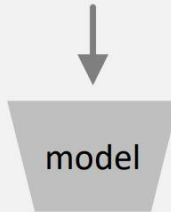
- “sample” $x \Rightarrow$ “label” y
- one desired output

generative

- “label” $y \Rightarrow$ “sample” x
- many possible outputs

discriminative

x



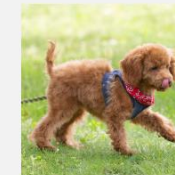
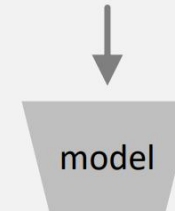
“dog”

y

generative

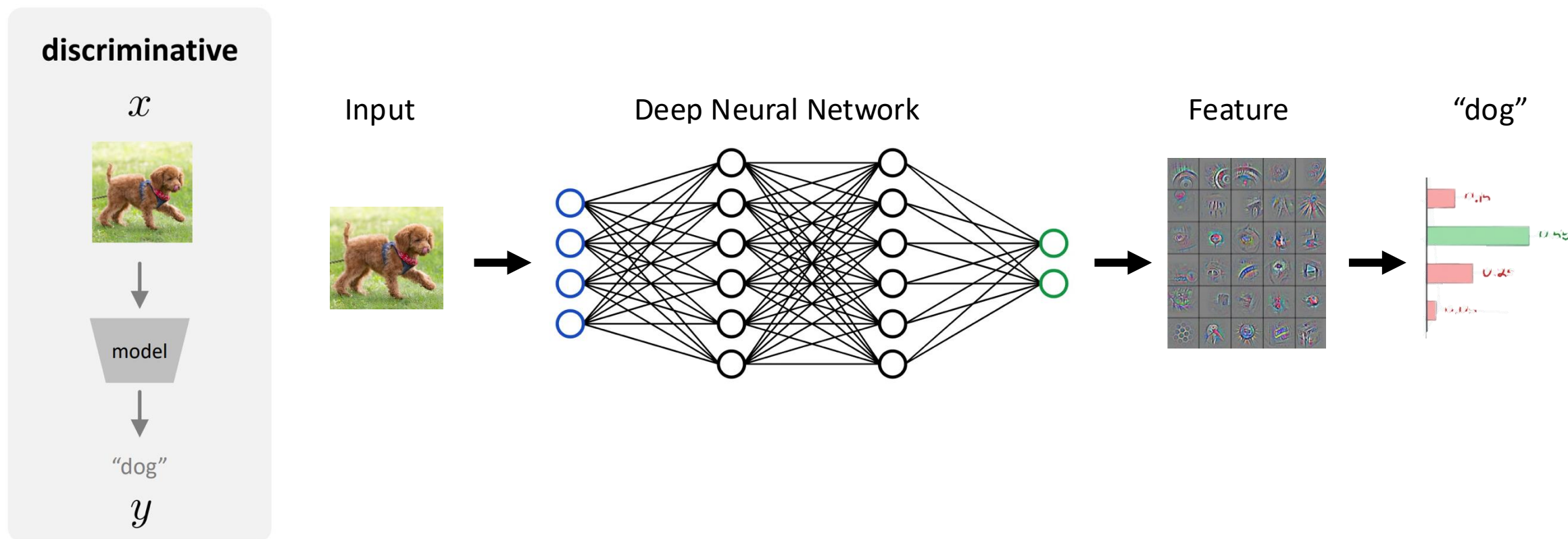
y

“dog”



x

Review of perception models



XOR (exclusive or) Problem

Marvin Minsky

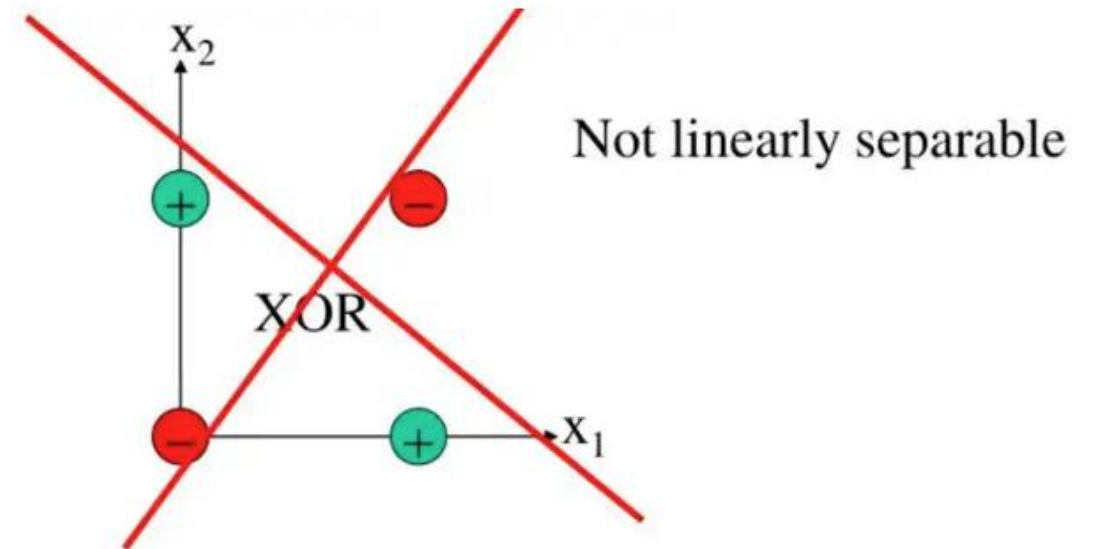
[Article](#) [Talk](#)

From Wikipedia, the free encyclopedia

Marvin Lee Minsky (August 9, 1927 — January 24, 2016) was an American [cognitive](#) and [computer scientist](#) concerned largely with research in [artificial intelligence](#) (AI). He co-founded the [Massachusetts Institute of Technology](#)'s AI laboratory and wrote extensively about AI and philosophy.^{[12][13][14][15]}

Minsky received many accolades and honors, including the 1969 ACM [Turing Award](#). He is known as the "father of AI".^[16]

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



Minsky & Papert (1969)

Perceptrons can only represent linearly separable functions.

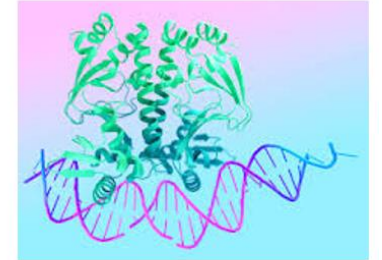
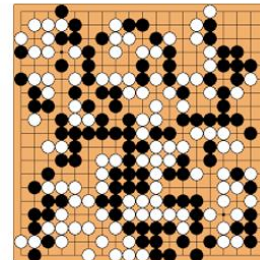
Machine Perception is Representation Learning

- Representation Learning: worth a conference name 😊
 - International Conference on Learning Representations (ICLR)
- Represent raw data for machines to solve complex problems
 - compression, abstraction, conceptualization
- Raw data in different forms
 - Vision: pixels, ...
 - Language: words, ...
 - Speech: waves, ...
 - Games: status, ...

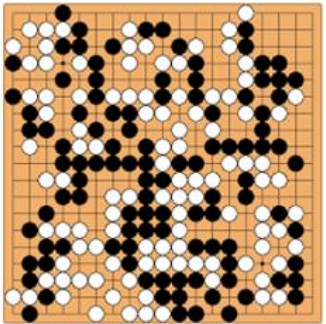


Welcome to all! Greetings President Faust, deans and professors, guests, friends, relatives, and fellow students! We have come together in this gem of the Harvard campus, which has become ours over the past years, about to begin a new stage in life's journey. Although we have studied different things and will pursue diverse paths, is there nonetheless something that we have all learned and of which we can all make use? Indeed I believe there is. I call it the heart of Harvard, our heart, that is enthusiasm.

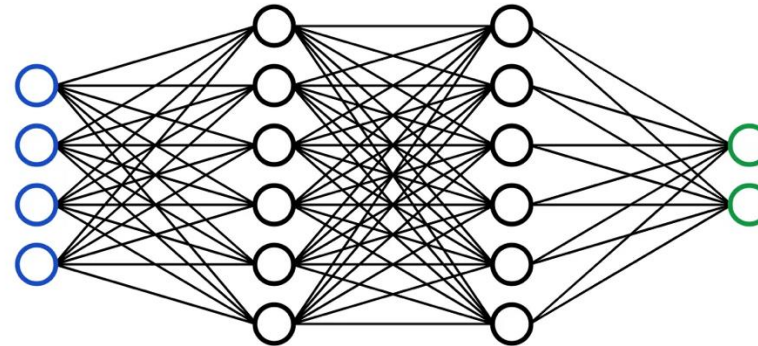
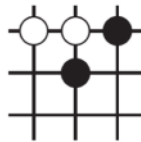
When I first arrived at Harvard, I did not know what to expect, beyond brilliant minds, a great deal of work, and my own intellectual growth. I was not disappointed. Many times, as I left class or office hours pleasantly exhausted mentally, I certainly marveled at the wisdom and talents of my professors and fellow students. But it was not Harvard's erudition that touched me over these past four years but a warmth and enthusiasm that I could not have anticipated.



Representation Learning



3^{361} states?



*Good
representations*



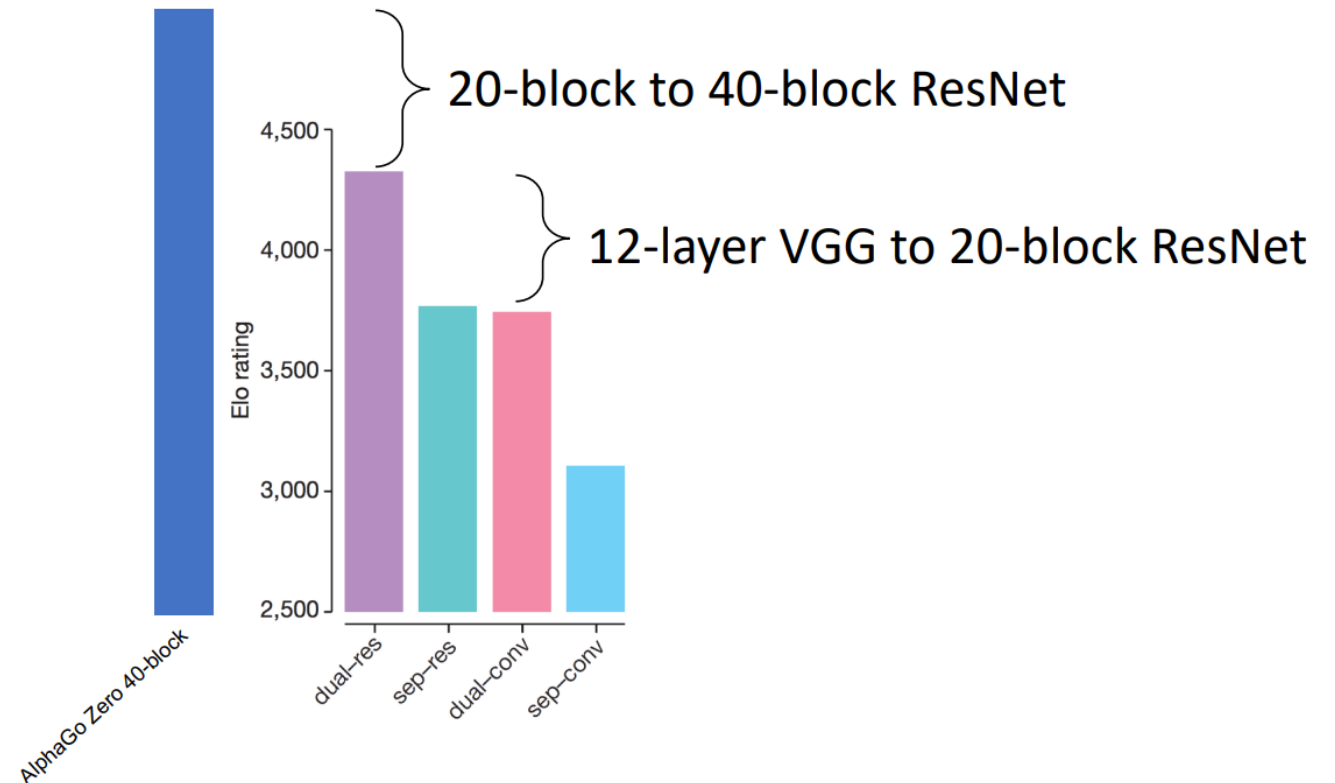
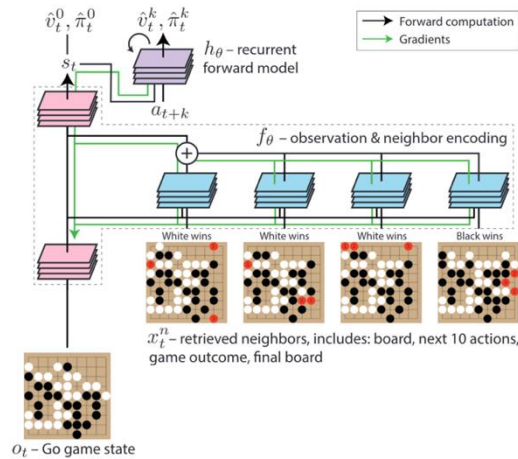
$256^3 \times 640 \times 480$?



Representation Learning

AlphaGo: better representation leads to higher rating

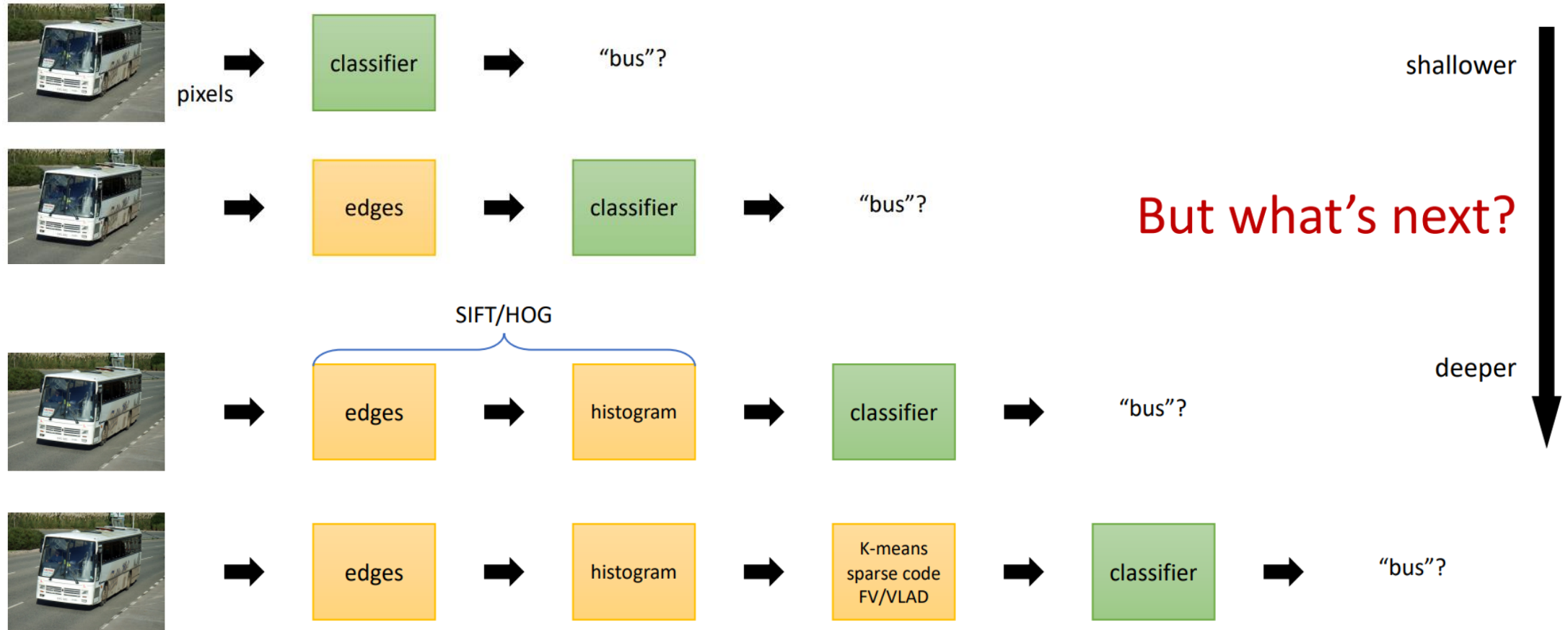
- outperform best human players
- without human knowledge



Vinyals, Oriol, et al. "Grandmaster level in StarCraft II using multi-agent reinforcement learning." *nature*.

Silver, David, et al. "Mastering the game of go without human knowledge." *nature* (2017).

How to represent an image?



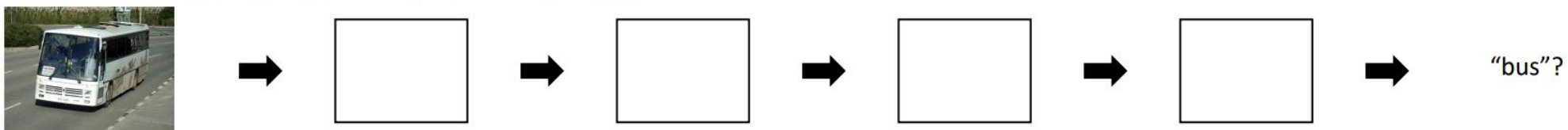
[Lowe 1999, 2004], [Sivic & Zisserman 2003], [Dalal & Triggs 2005], [Grauman & Darrell 2005]
[Lazebnik et al 2006], [Perronnin & Dance 2007], [Yang et al 2009], [Jégou et al 2010],

Learning to represent

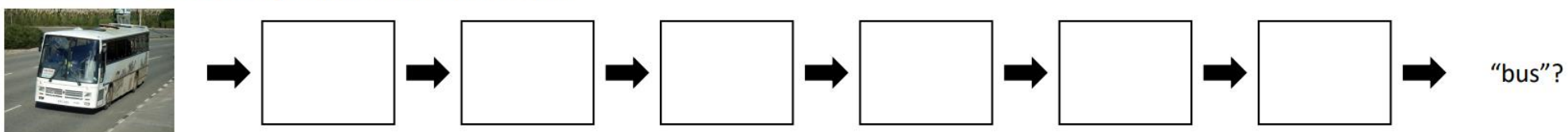
Specialized components, domain knowledge required



Generic components, less domain knowledge



Repeat **elementary** layers: going deeper

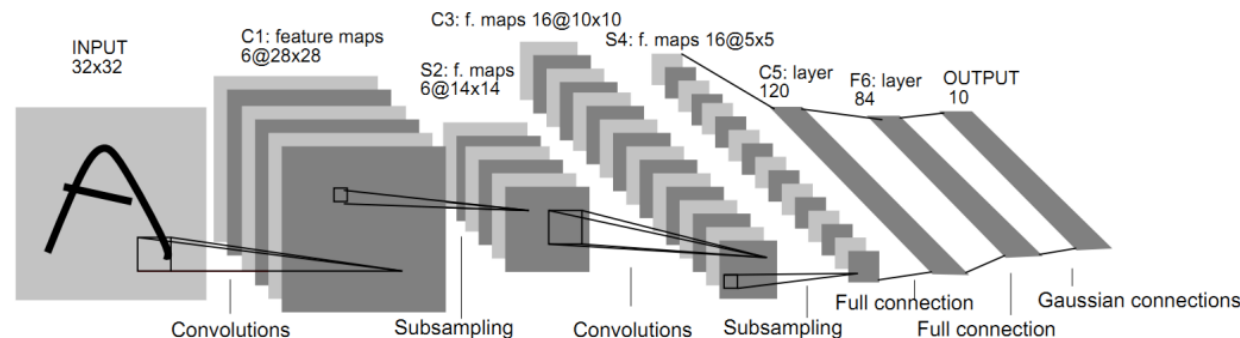


Compose simple modules into complex functions

- build multiple levels of abstractions
- learn by Backpropagation
- learn from data
- reduce domain knowledge and future engineering

Take image recognition as an example

LeNet (1989)



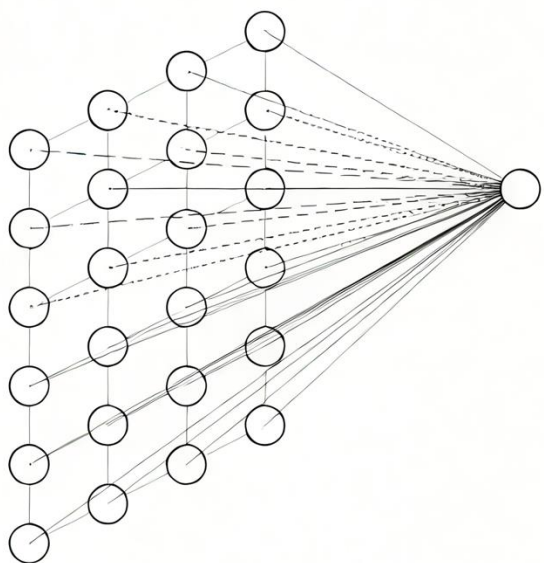
- Convolution:
 - locally-connected
 - spatially **weight-sharing**
 - weight-sharing is a key in DL (e.g., RNN shares weights temporally)
- Subsampling
- Fully-connected outputs
- Train by BackProp
- All are still the basic components of modern ConvNets!

“Backpropagation applied to handwritten zip code recognition”, LeCun et al. 1989

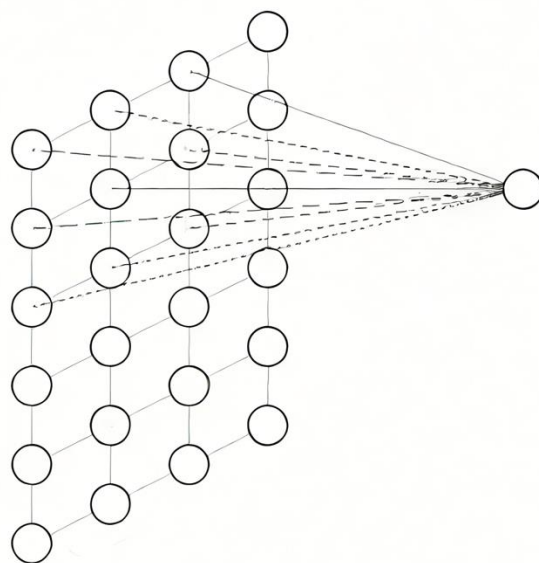
“Gradient-based learning applied to document recognition”, LeCun et al. 1998

Convolution

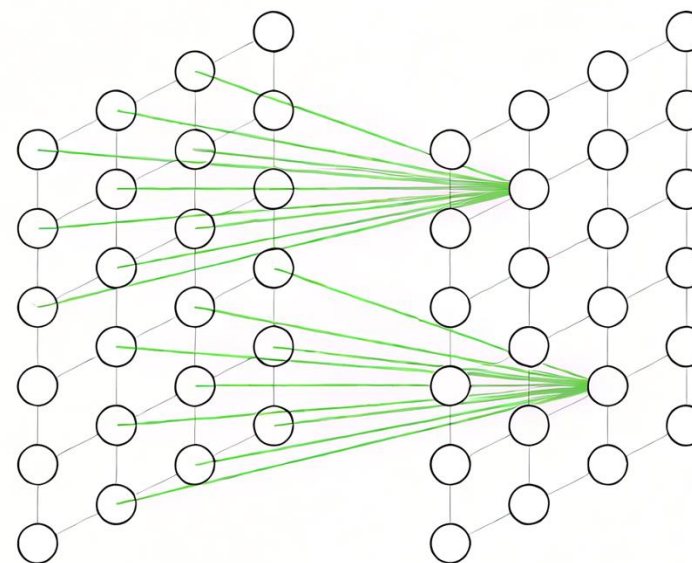
- convolution = local connections + spatial **weight-sharing**



fully-connected



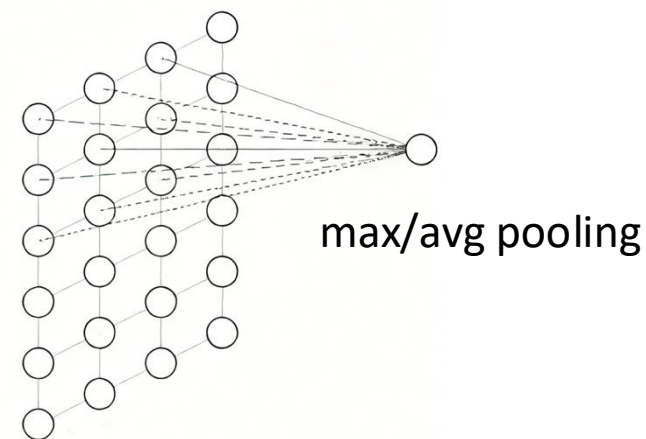
locally-connected



convolution

Pooling

- sub-sampling: produce small feature map
- achieve local invariance
- more abstract representation



One Feature Map

2	3	2	0
5	-2	2	8
-1	-6	7	3
-4	-5	4	2

Pooling

5	8
-1	7

(a) Max-Pooling

One Feature Map

2	3	2	0
5	-2	2	8
-1	-6	7	3
-4	-5	4	2

Pooling

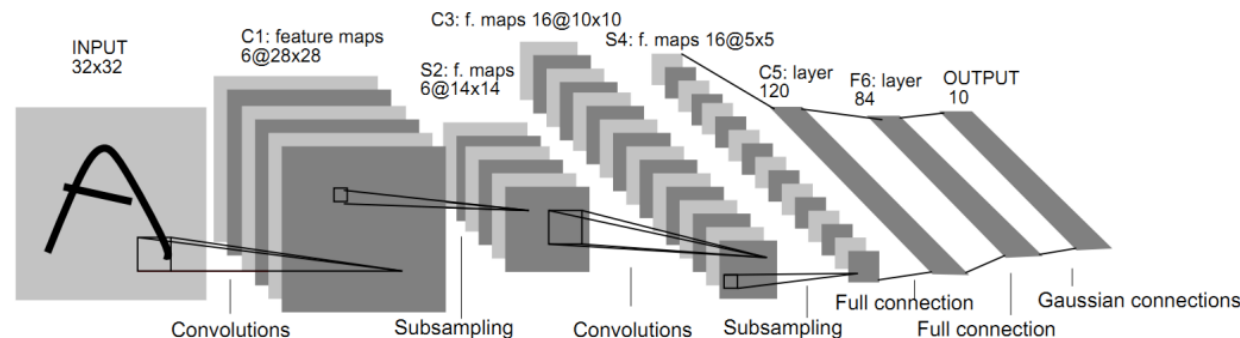
2	3
-4	4

(b) Average-Pooling



32 5×5 kernel, 6 channels 182×62 kernel, 16 channels 10×10 kernel, 16 channels 15×15 kernel, 16 channels

LeNet (1989)



Convolutional Neural Network

- convolution
- pooling
- fully-connected layer
- trained by backpropagation

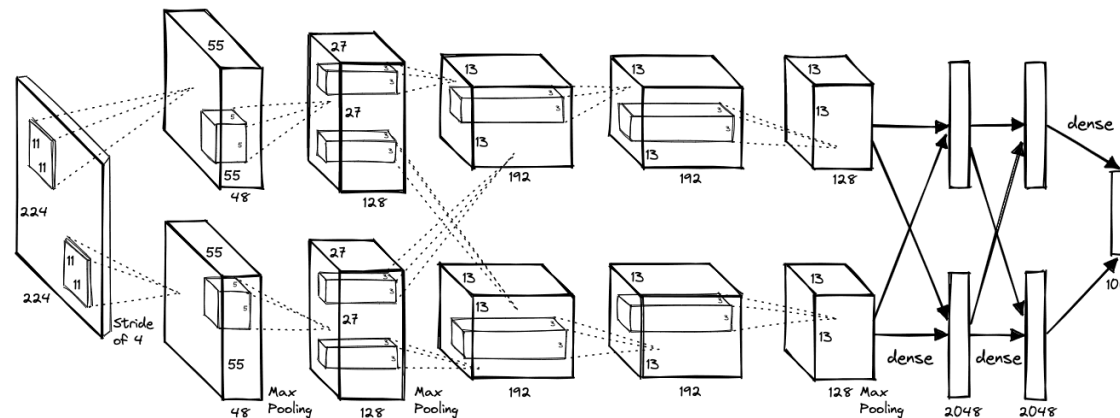
Data: MNIST

- 50,000 images, 10 classes

“Backpropagation applied to handwritten zip code recognition”, LeCun et al. 1989

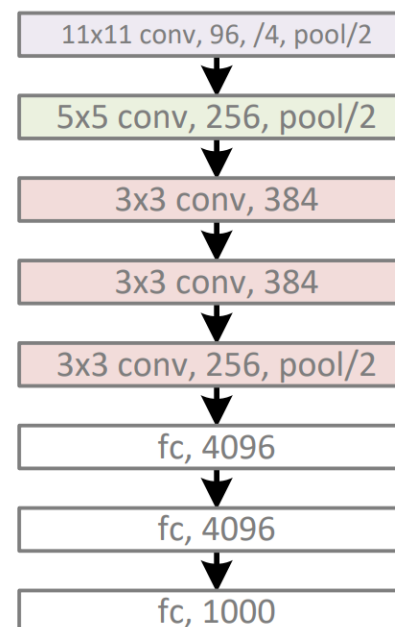
“Gradient-based learning applied to document recognition”, LeCun et al. 1998

AlexNet (2012)



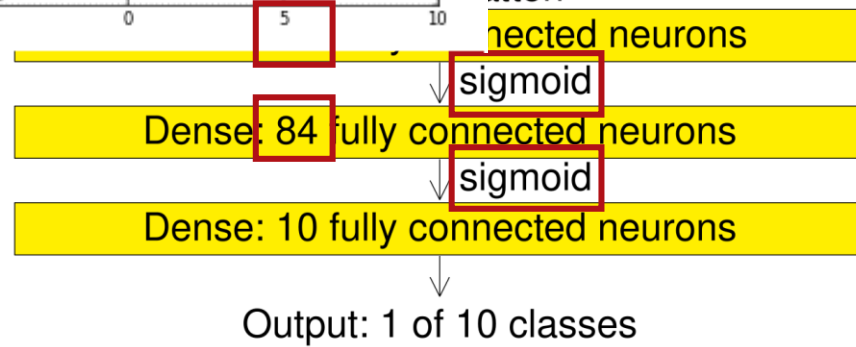
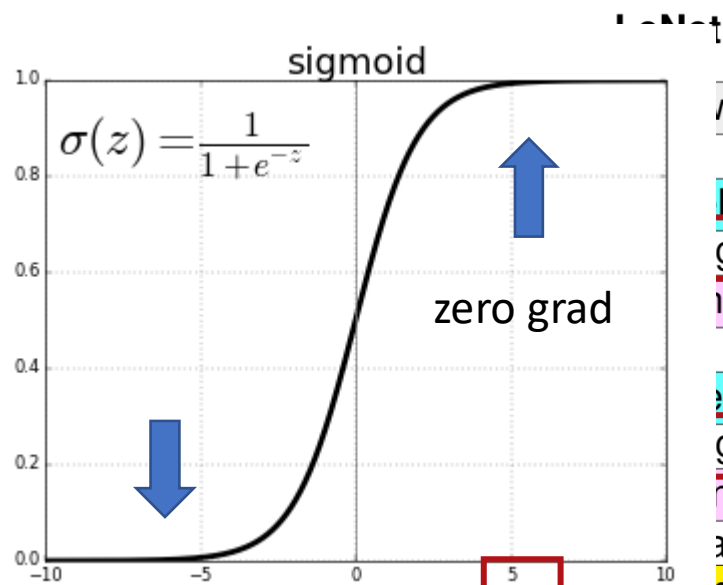
LeNet-style backbone, plus:

- ReLU [Nair & Hinton 2010]
 - “**RevoLU**tion of deep learning”*
 - Accelerate training; better grad prop (vs. tanh)
- Dropout [Hinton et al 2012]
 - In-network ensembling
 - Reduce overfitting (might be instead done by BN)
- Data augmentation
 - Label-preserving transformation
 - Reduce overfitting



*Quote Christian Szegedy

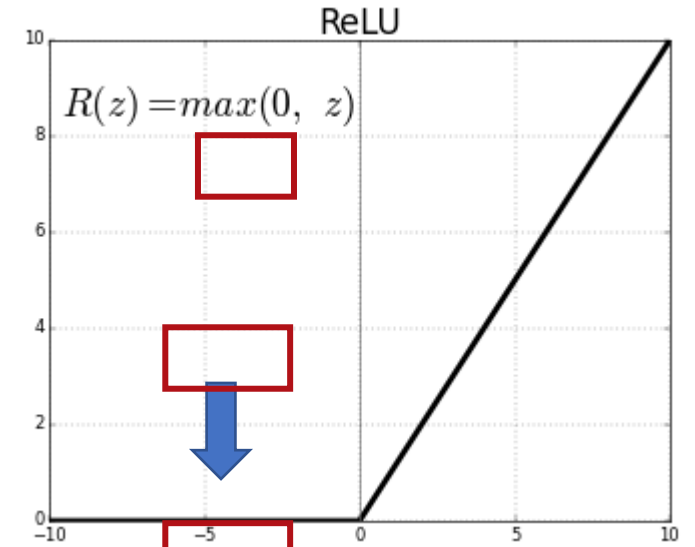
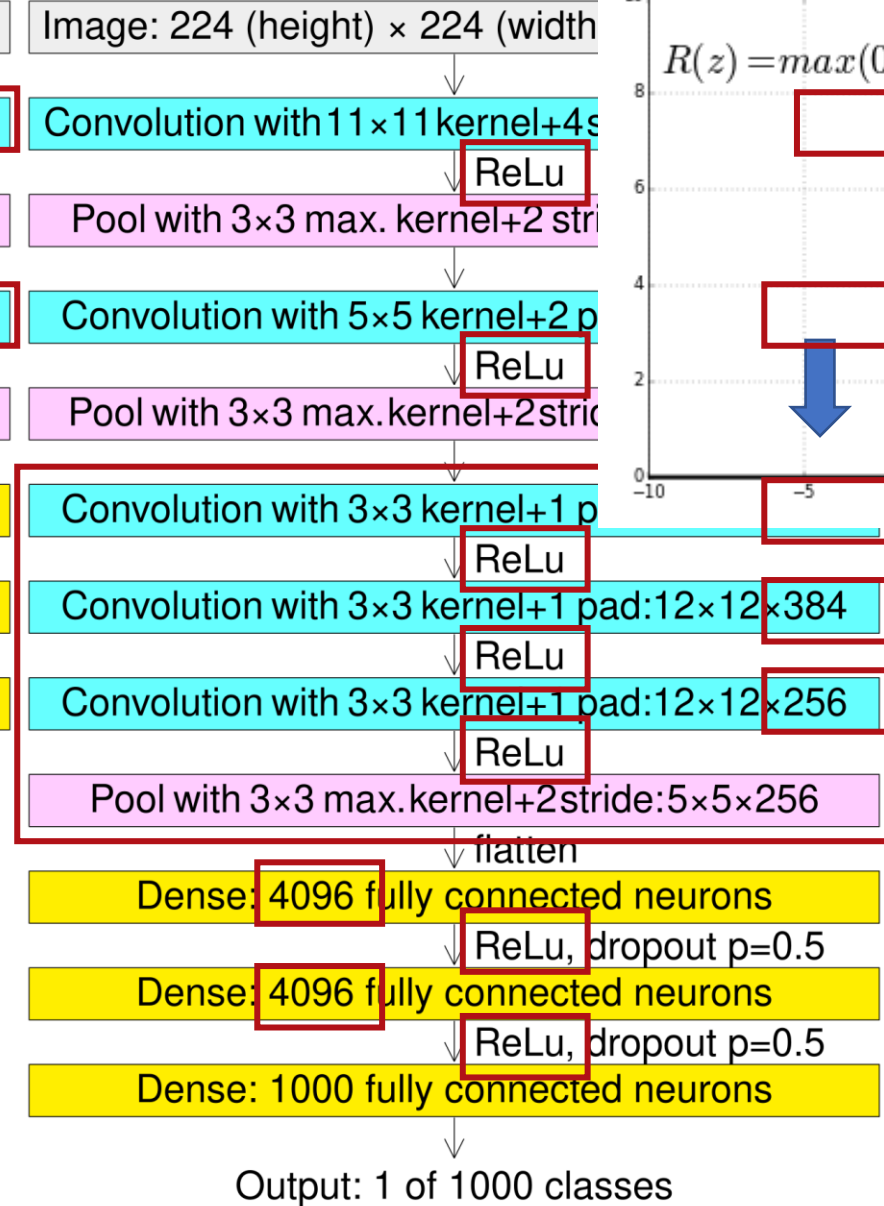
- Scaling up ConvNet
 - Data scaling (ImageNet)
 - 1.28 million images
 - 1000 classes
 - Model scaling
 - 60 million parameters
 - 650,000 neurons
 - GPU training
 - data distribution
 - model distribution



Deep rectified linear unit

- higher level of abstraction
- stronger representation network

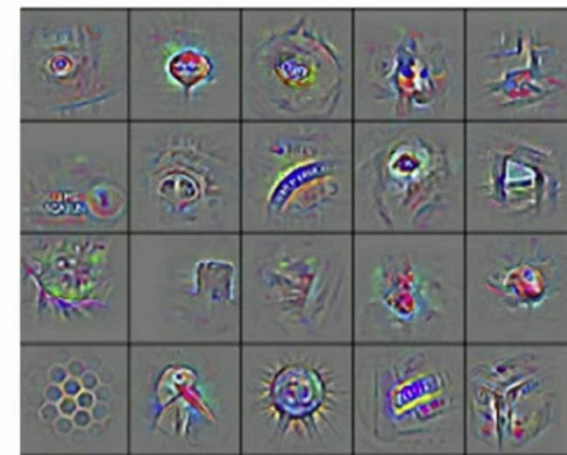
AlexNet



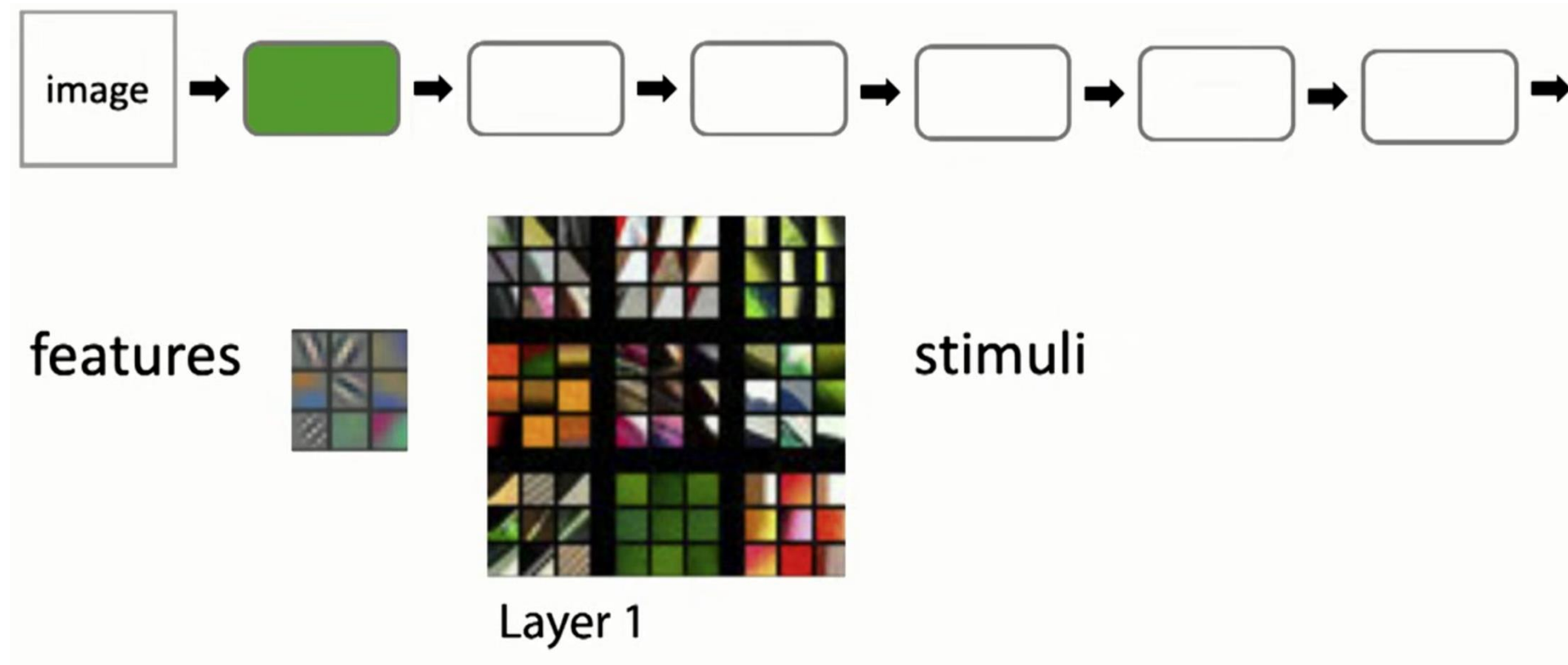
2013: Visualizing ConvNet

“Seeing is understanding”

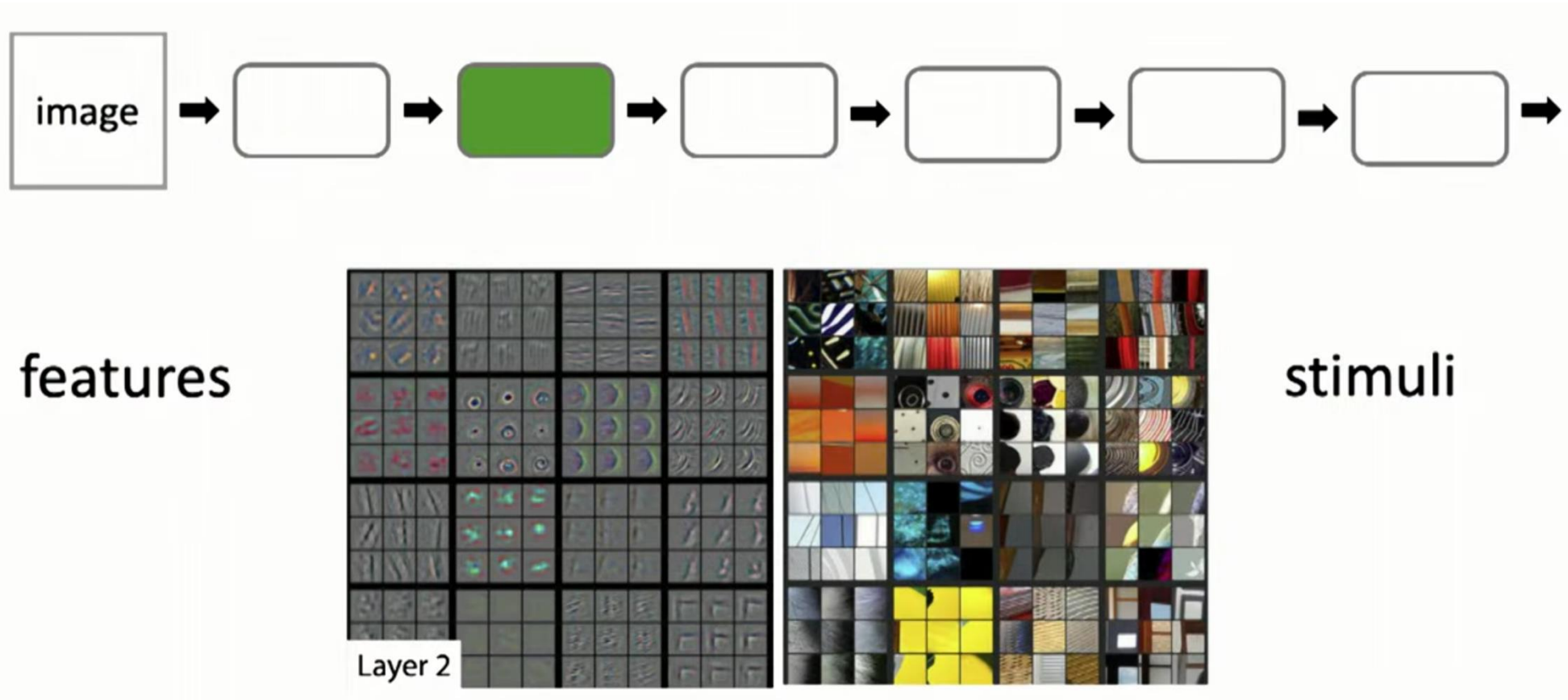
- Understand representations by visualization
- Visualization: to find “what input can produce the feature”
 - set a one-hot feature map
 - back-prop to pixels



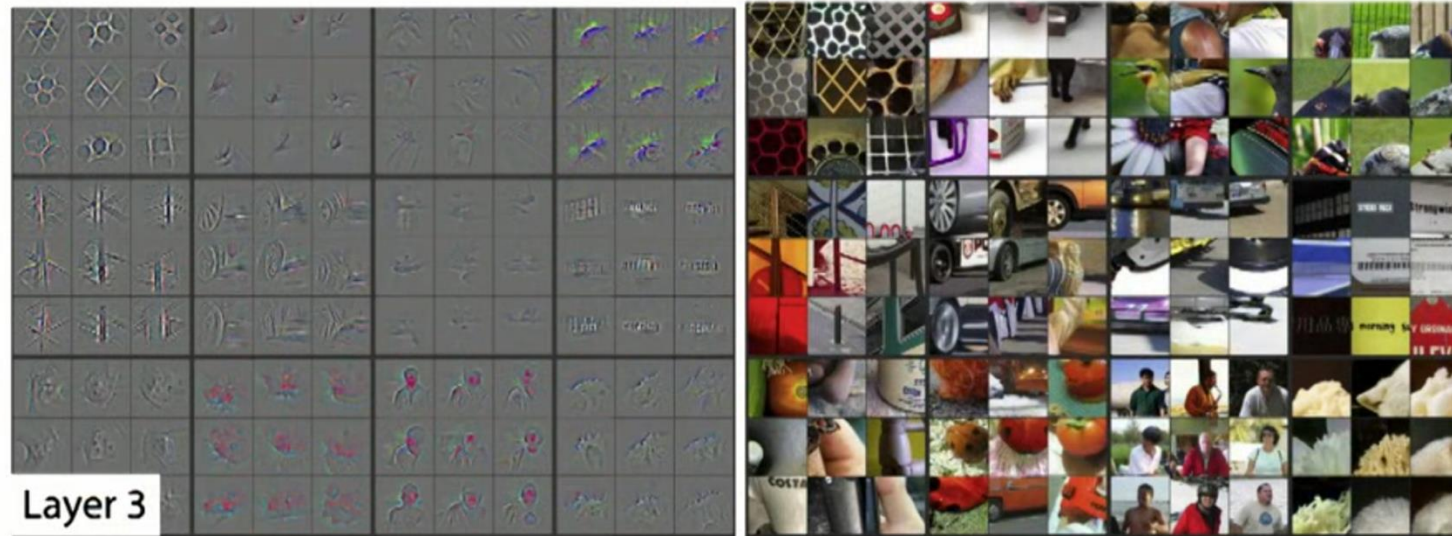
Visualizations



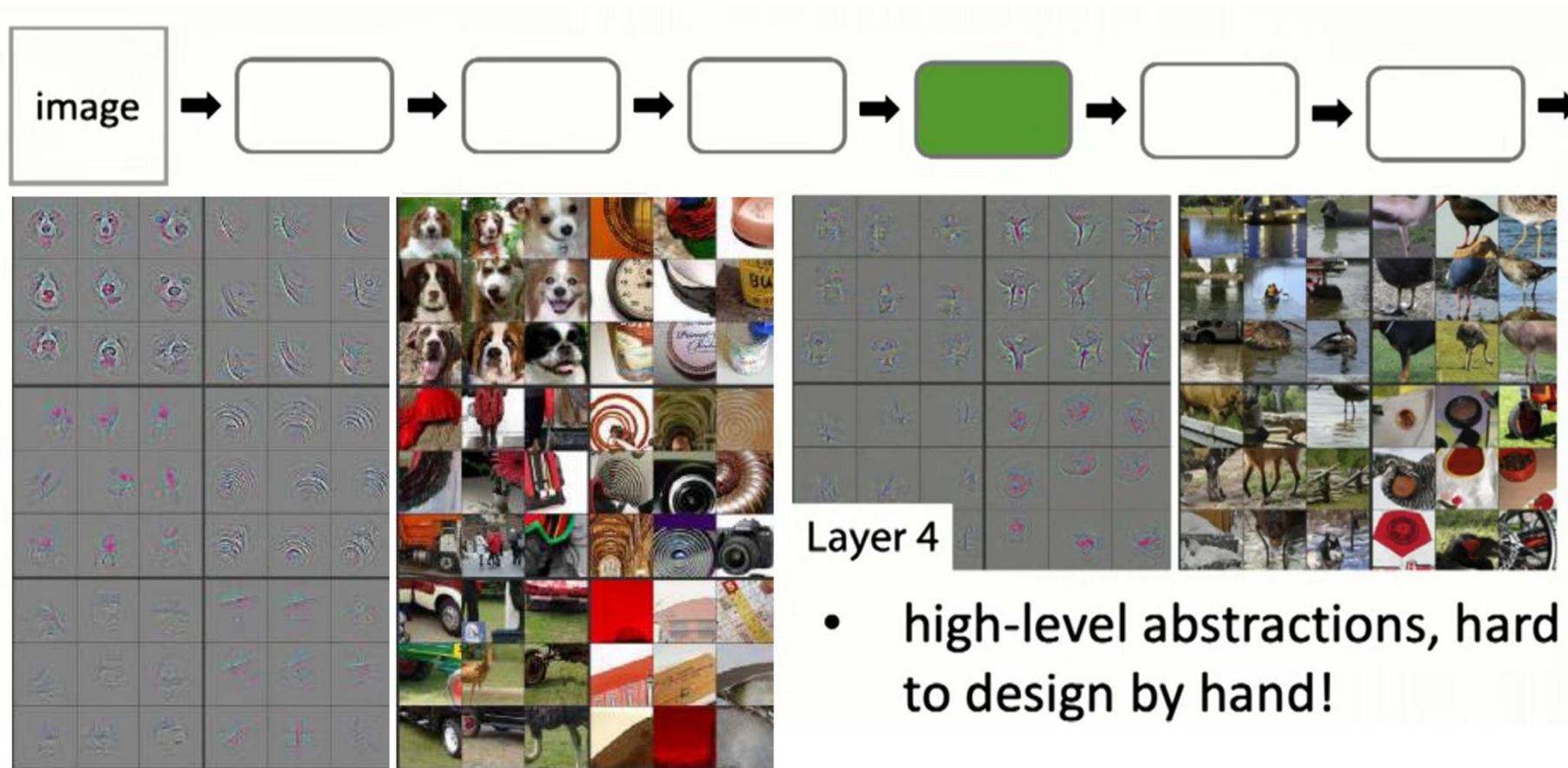
Visualizations



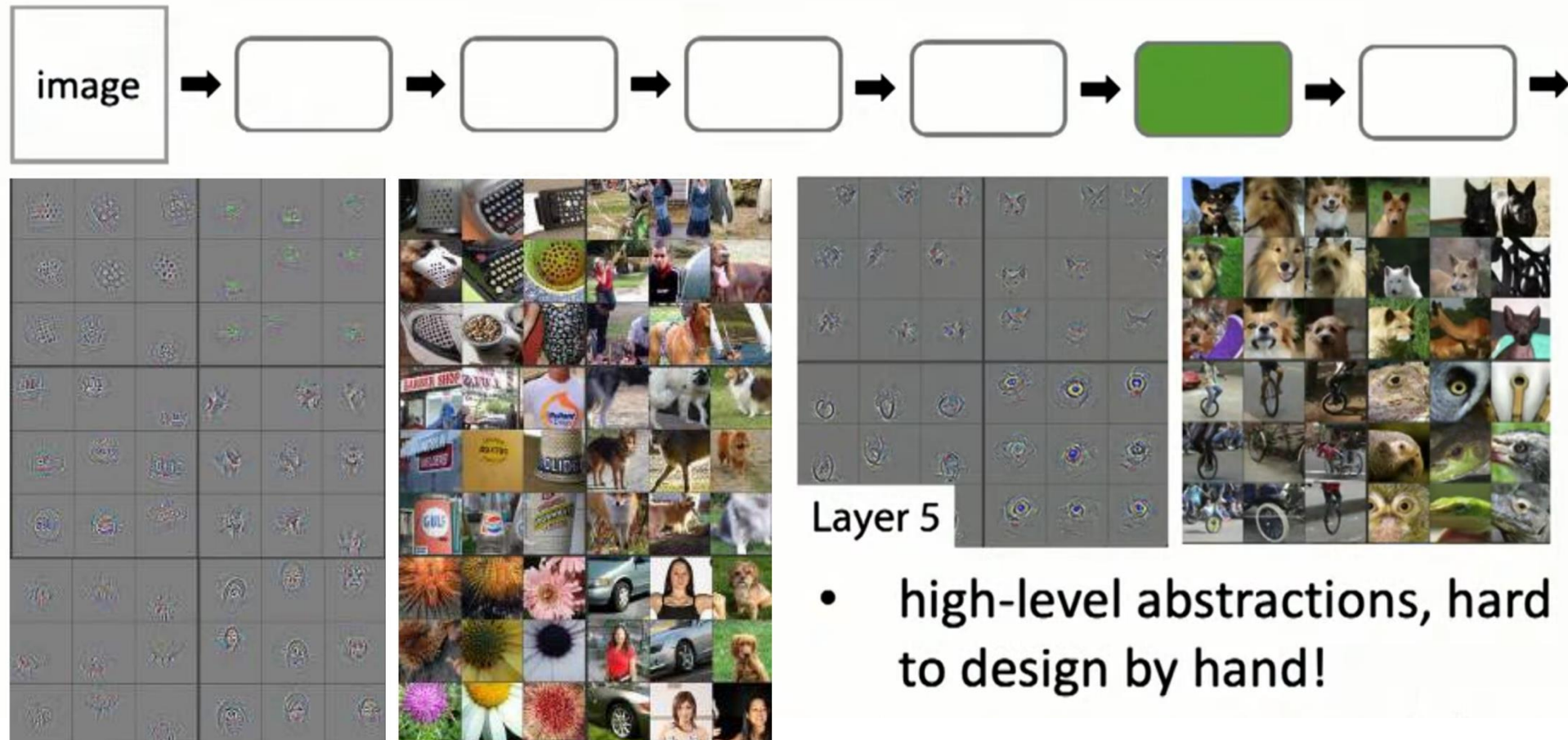
Visualizations



Visualizations



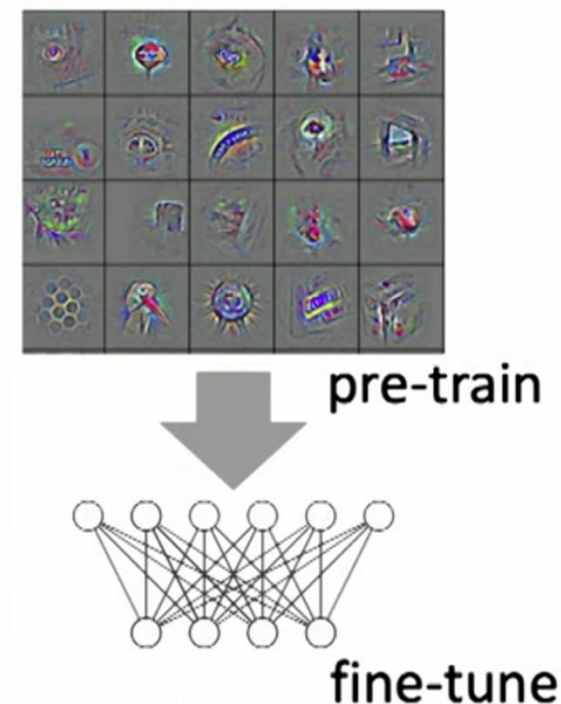
Visualizations



Deep Representations are Transferrable!

The single most important discovery in DL revolution

- Transfer learning
 - pre-train on large-scale data
 - fine-tune on small-scale data
- enable DL for small datasets
- revolutionize computer vision
- ImageNet: engine for general representation
- GPT: a similar principle



2014: VGG Net

“16 layers are beyond my imagination!”

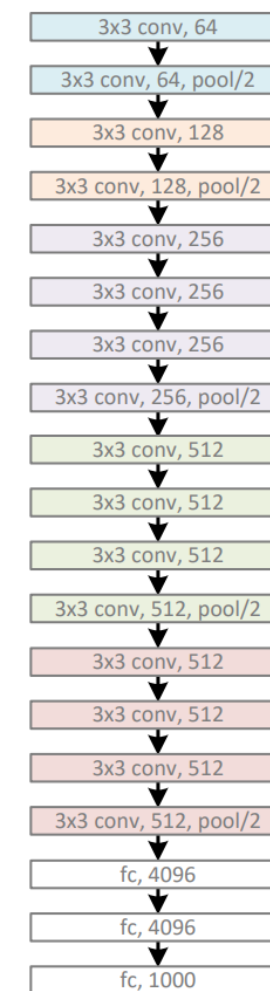
-- after ILSVRC 2014 result was announced.



HONG KONG

Simply “Very Deep”!

- Modularized design
 - 3x3 Conv as the module
 - Stack the same module
 - Same computation for each module (1/2 spatial size => 2x filters)
- Stage-wise training
 - VGG-11 => VGG-13 => VGG-16
 - We need a better initialization...



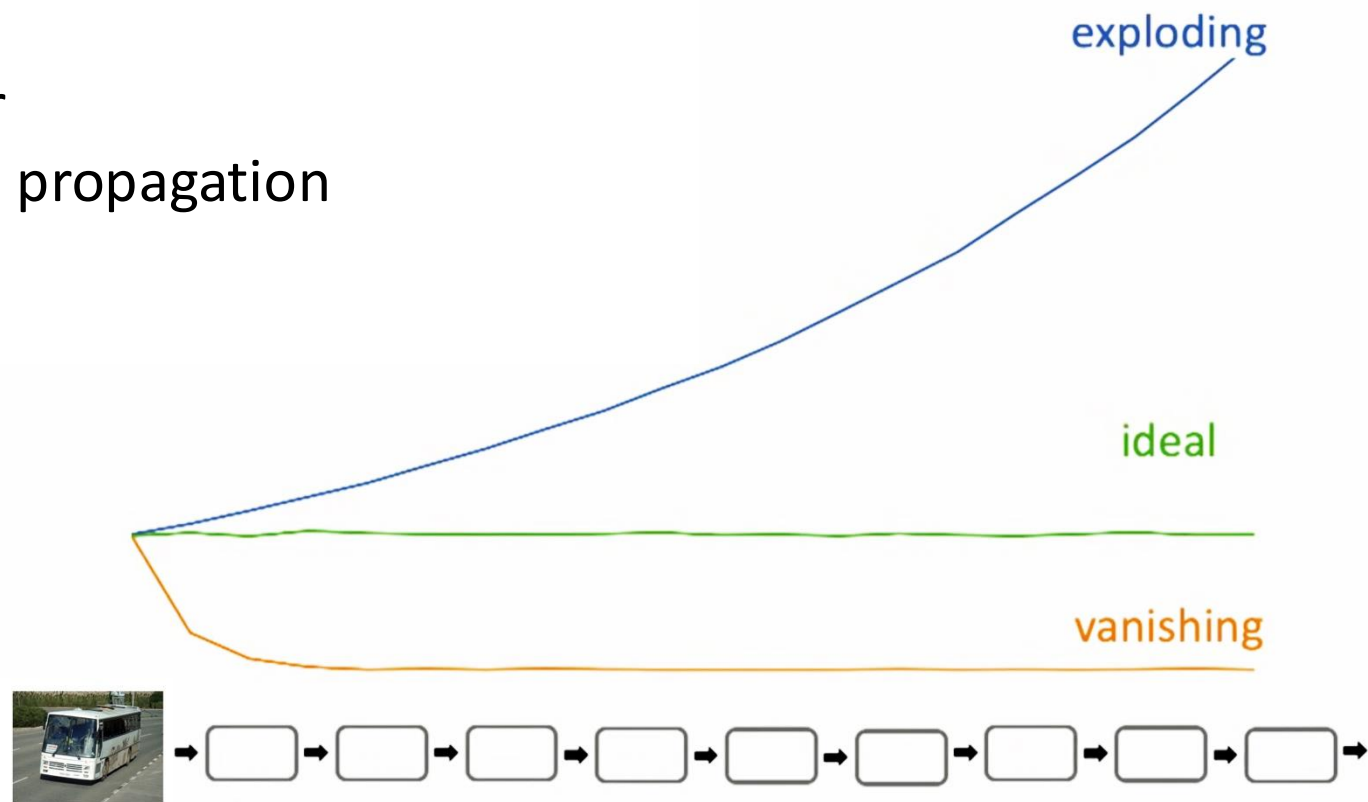
2014: VGG Net

- previous: little clear evidence
- controlled experiments
 - remove bells-and-whistles
 - little “config engineering”
 - simple rules of setting channels
 - add 3x3 conv
- simple but hard-to-beat
 - before deeper nets come along
- stage-wise training ☹️
 - 11 -> 13 -> 16 -> 19 layers
 - better initialization needed

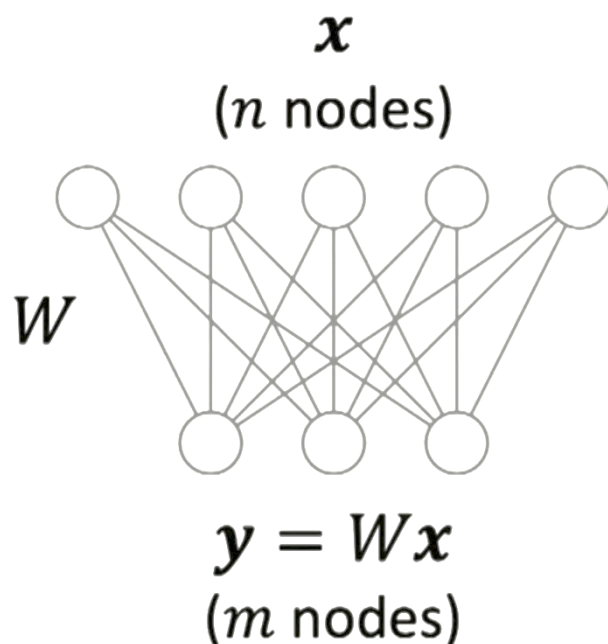
11	13	16	19		
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
max pool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
max pool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
max pool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
max pool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
max pool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

2015: Network initialization

- difficulties of going deeper
 - troubles accumulate in propagation
 - exploding signals
 - vanishing signals



2015: Network initialization



- if x, y, w have independent elements
- if linear activation
- one layer: variance scaled by

$$\text{Var}[\mathbf{y}] = n \text{Var}[\mathbf{w}] \text{Var}[\mathbf{x}]$$

- many layers: variance scaled by

$$\text{Var}[\mathbf{y}] = \prod_d n_d \text{Var}[\mathbf{w}_d] \text{Var}[\mathbf{x}]$$

2015: Network initialization

forward

$$\text{Var}[y] = \prod_d n_d \text{Var}[w_d] \text{Var}[x]$$

backward

$$\text{Var} \left[\frac{\partial \mathcal{E}}{\partial x} \right] = \prod_d m_d \text{Var}[w_d] \text{Var} \left[\frac{\partial \mathcal{E}}{\partial y} \right]$$

vanishing gradient, if < 1

Initialization Methods

- Analytical formulations of normalizing forward/backward signals
- Based on strong assumptions (like Gaussian distributions)
- Xavier Init (linear): $n \cdot \text{Var}[w] = 1$
 - `torch.nn.init.xavier_uniform_(tensor, gain=1.0, generator=None)`
- Kaiming Init (ReLU): $n \cdot \text{Var}[w] = 2$
 - `torch.nn.init.kaiming_uniform_(tensor, a=0, mode='fan_in', nonlinearity='leaky_relu', generator=None)`

“Efficient Backprop”, LeCun et al, 1998

“Understanding the difficulty of training deep feedforward neural networks” Glorot & Bengio, 2010

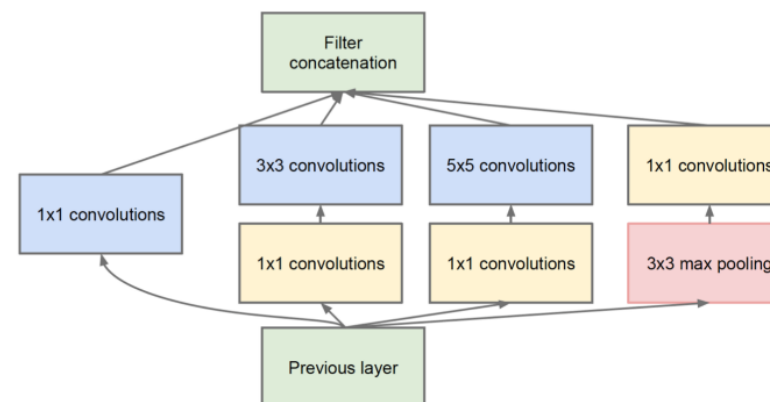
“Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification” He et al, ICCV 2015

2014: GoogleNet/Inception

Accurate with small footprint.

My take on GoogleNets:

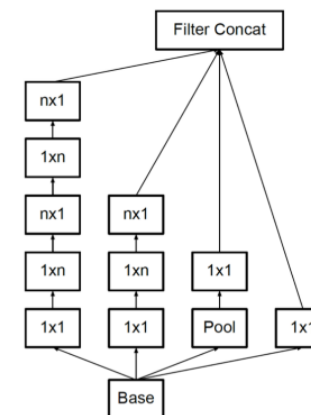
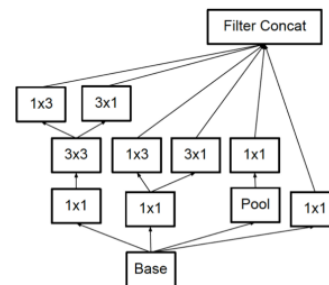
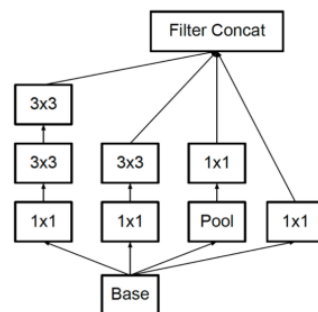
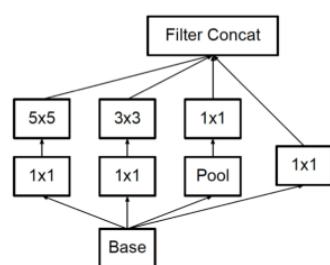
- Multiple branches
 - e.g., 1x1, 3x3, 5x5, pool
- Shortcuts
 - stand-alone 1x1, merged by concat.
- Bottleneck
 - Reduce dim by 1x1 before expensive 3x3/5x5 conv



GoogleNet/Inception v1, v2, v3, ...

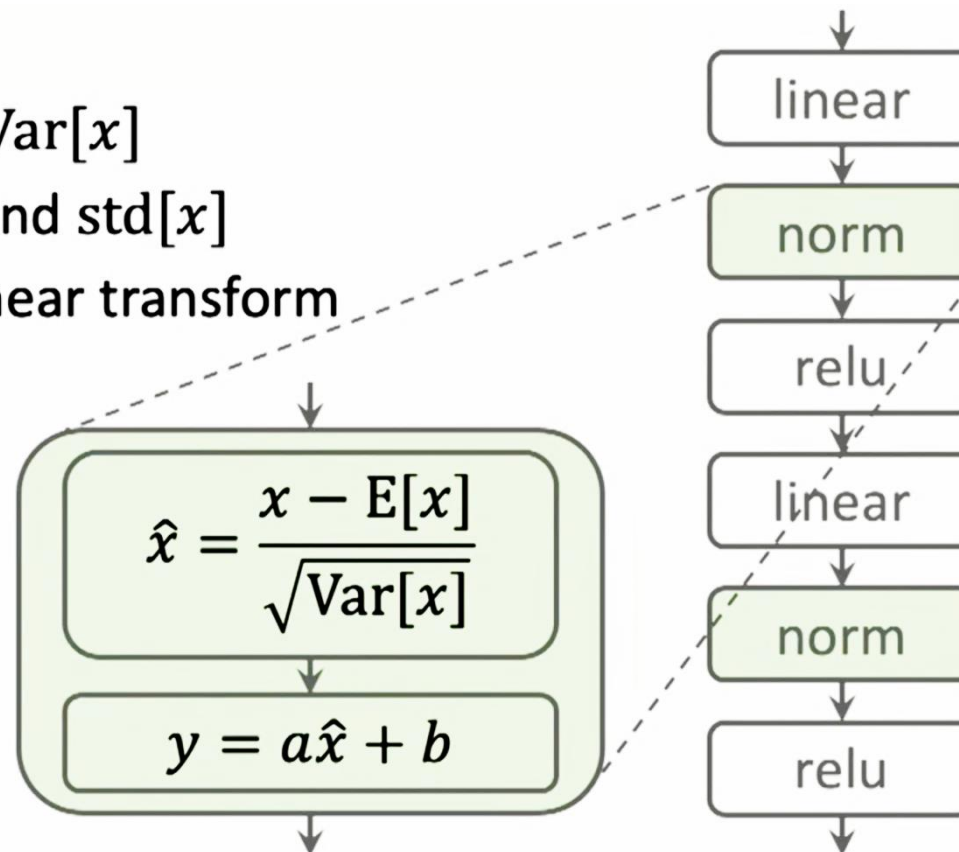
More templates, but the same 3 main properties are kept:

- Multiple branches
- Shortcuts (1x1, concat.)
- Bottleneck



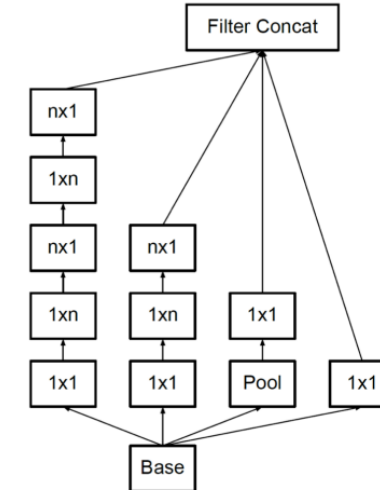
Normalization modules

- compute $E[x]$ and $\text{Var}[x]$
- normalize by $E[x]$ and $\text{std}[x]$
- compensate by a linear transform



BN: Batch Normalization

- Xavier/MSRA init are not directly applicable for multi-branch nets
- Optimizing multi-branch ConvNets largely benefits from BN
 - including all Inceptions and ResNets



BN: Batch Normalization

- Recap: Normalizing image input (LeCun et al 1998 “Efficient Backprop”)
- Xavier/MSRA init: Analytic normalizing each layer
- BN: data-driven normalization, **for each layer, for each mini-batch**
 - Greatly accelerate training
 - Less sensitive to initialization
 - Improve regularization

BN: Batch Normalization

$$x \rightarrow \hat{x} = \frac{x - \mu}{\sigma} \rightarrow y = \gamma \hat{x} + \beta$$

- μ : mean of x in **mini-batch**
- σ : std of x **in mini-batch**
- γ : scale
- β : shift
- μ, σ : functions of x , analogous to responses
- γ, β : parameters to be learned, analogous to weights

BN: Batch Normalization

$$x \rightarrow \hat{x} = \frac{x - \mu}{\sigma} \rightarrow y = \gamma \hat{x} + \beta$$

2 modes of BN:

- Train mode:
 - μ, σ are functions of a batch of x
- Test mode:
 - μ, σ are pre-computed on training set

Caution: make sure your BN usage is correct!
(this causes many of my bugs in my research experience!)

BN: Batch Normalization

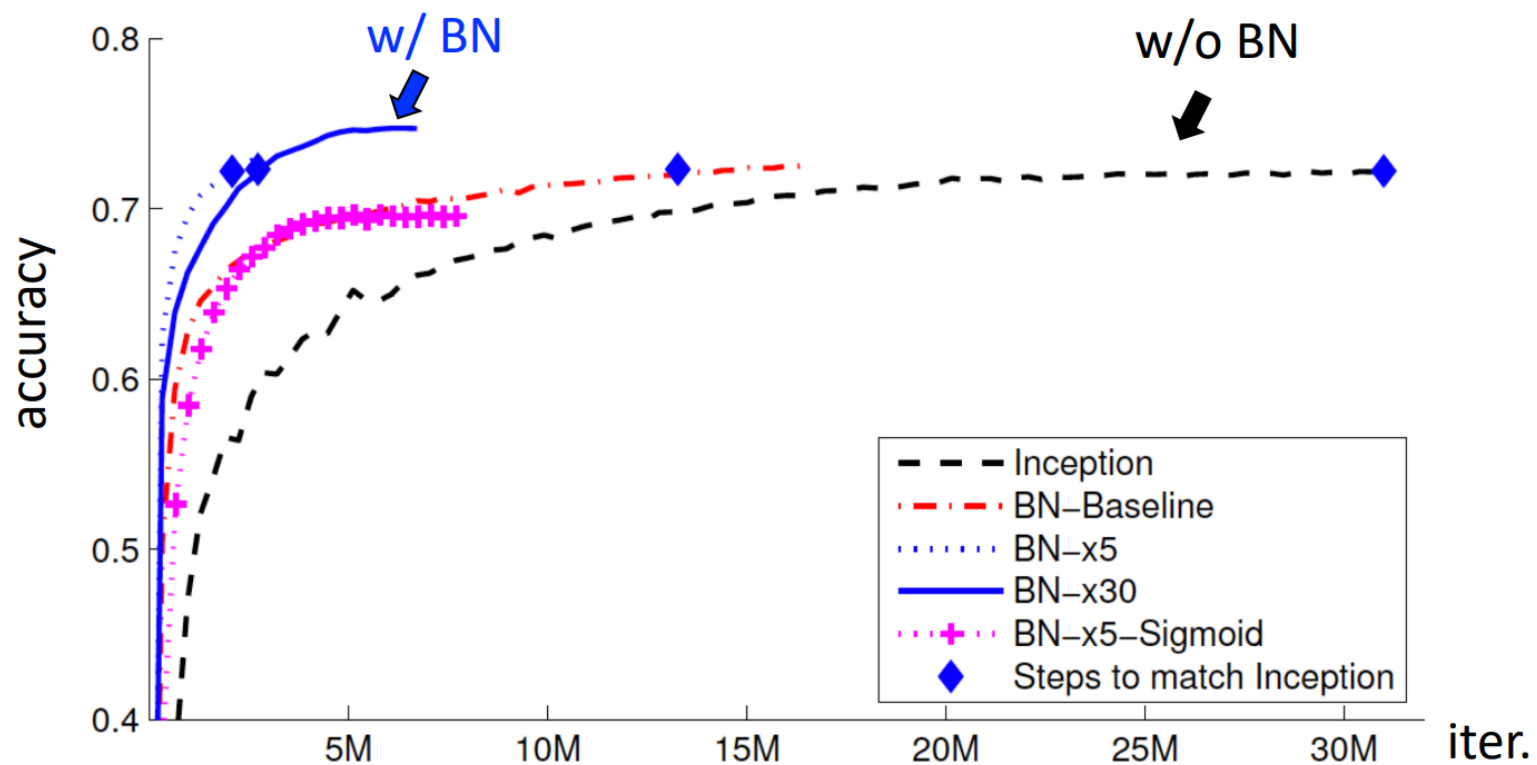
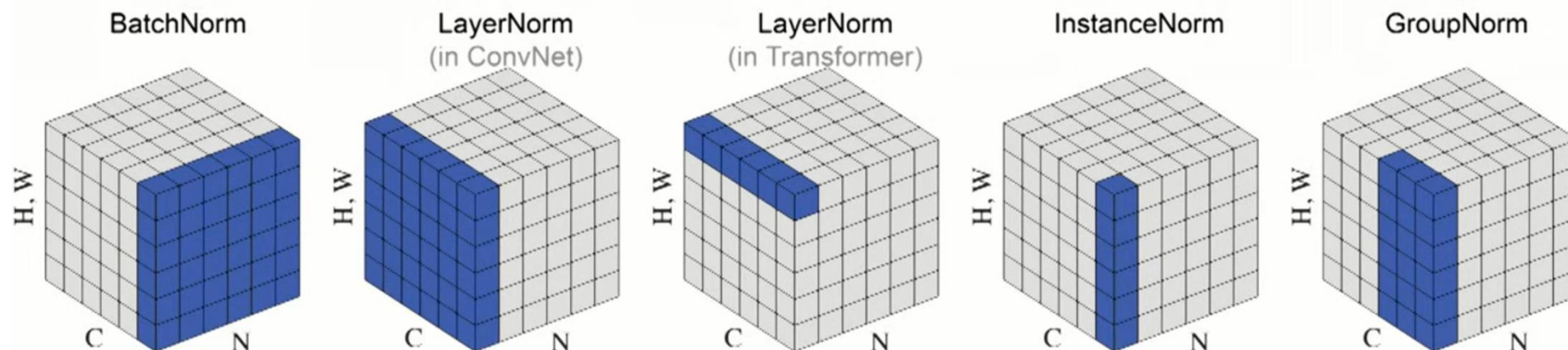


Figure credit: Ioffe & Szegedy

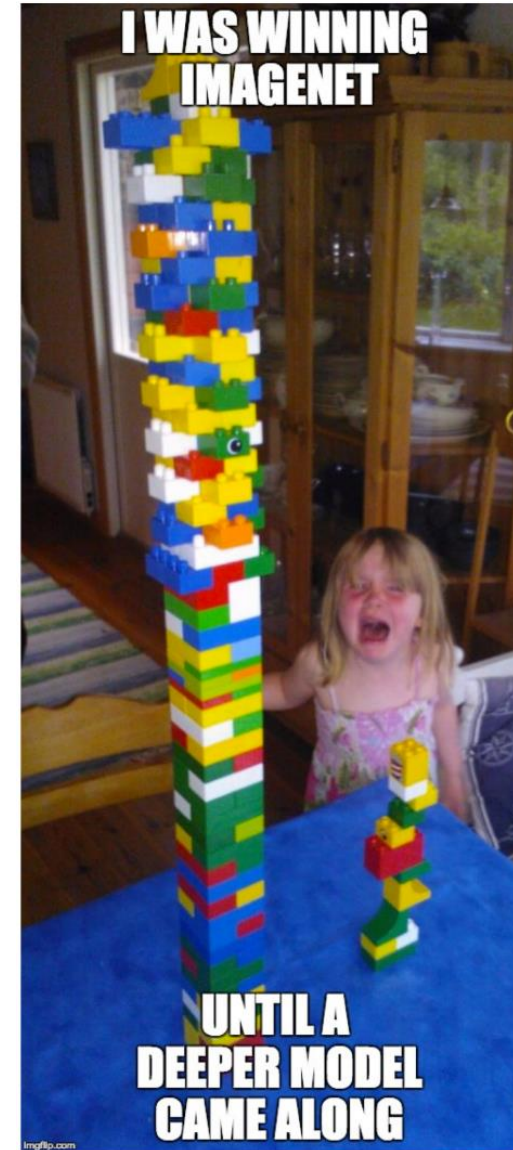
Variants of normalization modules

based on **support set** of $E[x]$ and $\text{Var}[x]$



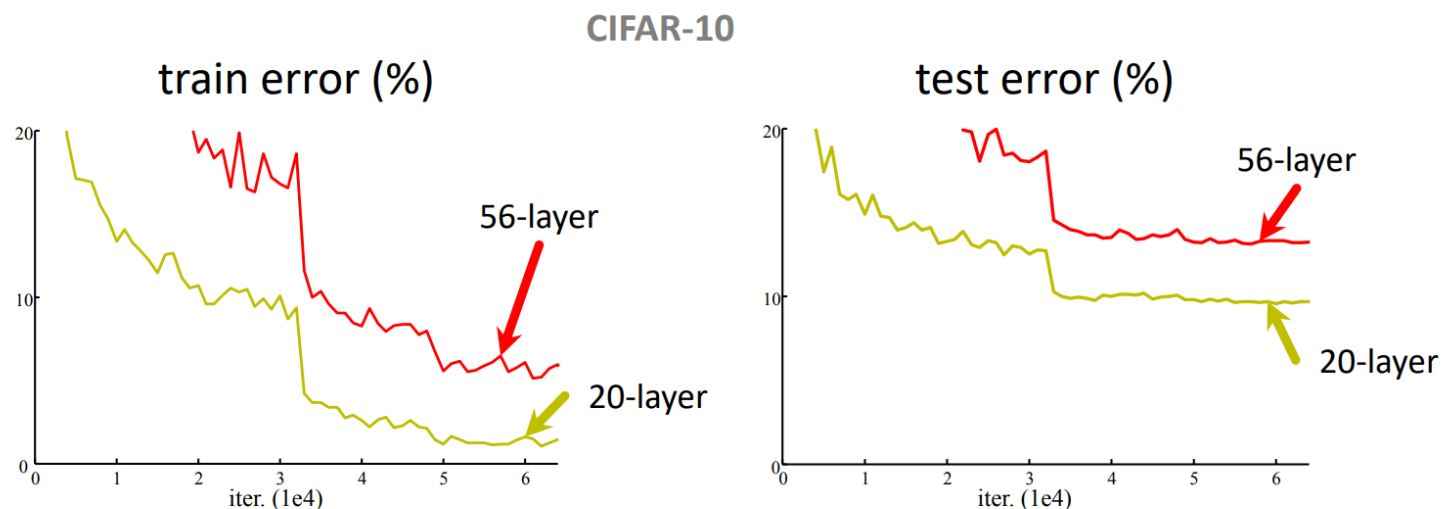
2015: ResNet

- Recap:
 - Compose simple modules into complex functions
- Deep Residual Learning
 - Deep Learning gets way deeper
 - simple component: identity shortcut
 - enable networks w/ hundreds of layers



2015: ResNet

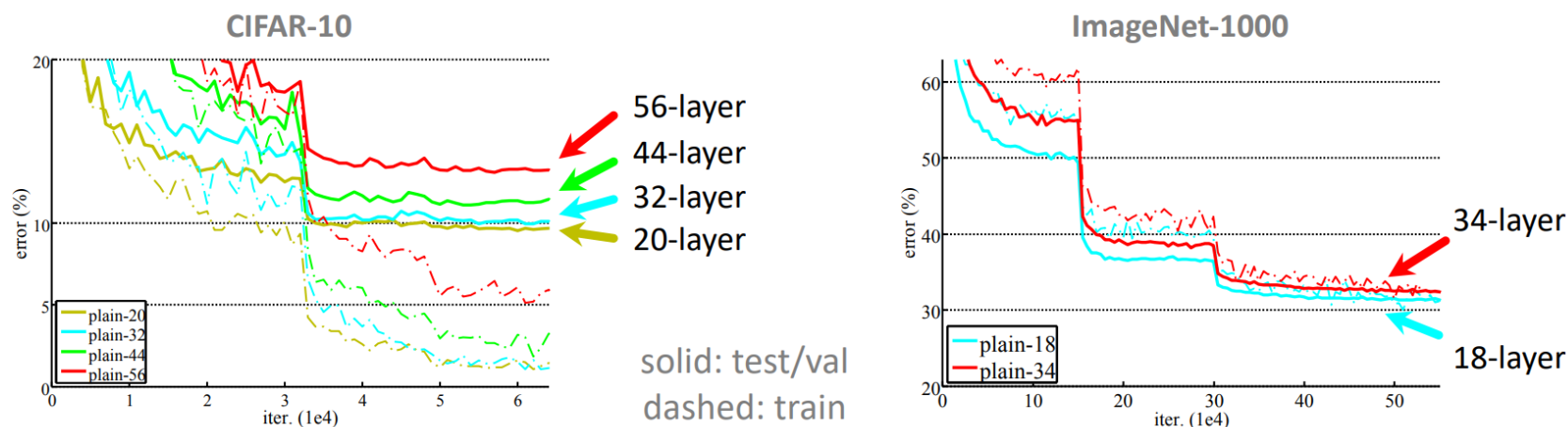
What about simply stacking layers?



- *Plain* nets: stacking 3x3 conv layers...
- 56-layer net has **higher training error** and test error than 20-layer net

2015: ResNet

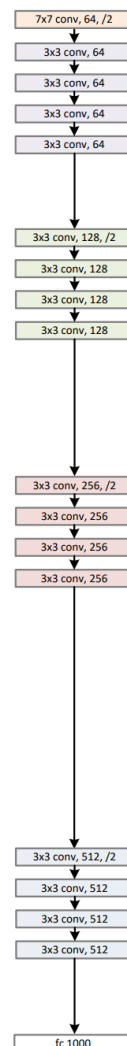
What about simply stacking layers?



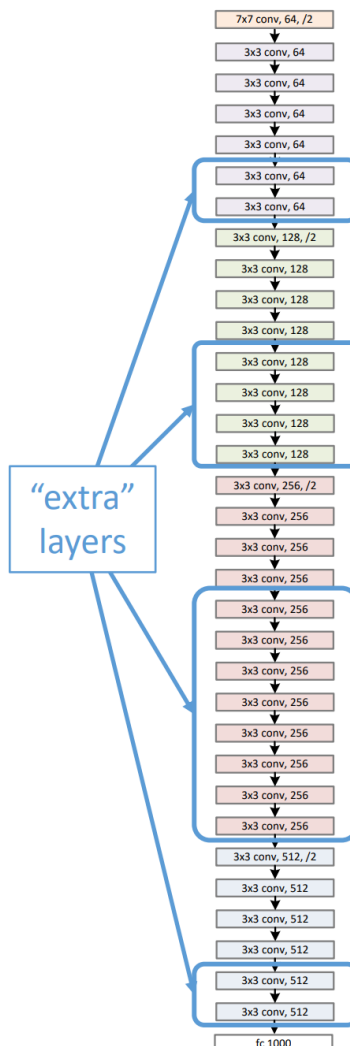
- “Overly deep” plain nets have **higher training error**
- A general phenomenon, observed in many datasets

2015: ResNet

a shallower
model
(18 layers)



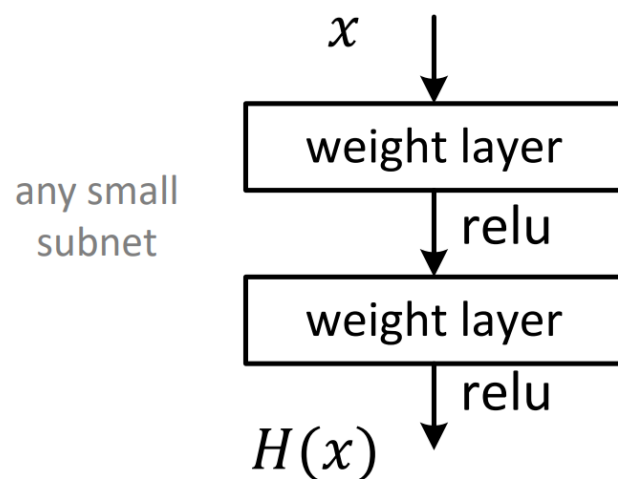
a deeper
counterpart
(34 layers)



- Richer solution space
- A deeper model should not have **higher training error**
- A solution *by construction*:
 - original layers: copied from a learned shallower model
 - extra layers: set as **identity**
 - at least the same training error
- **Optimization difficulties**: solvers cannot find the solution when going deeper...

Deep Residual Learning

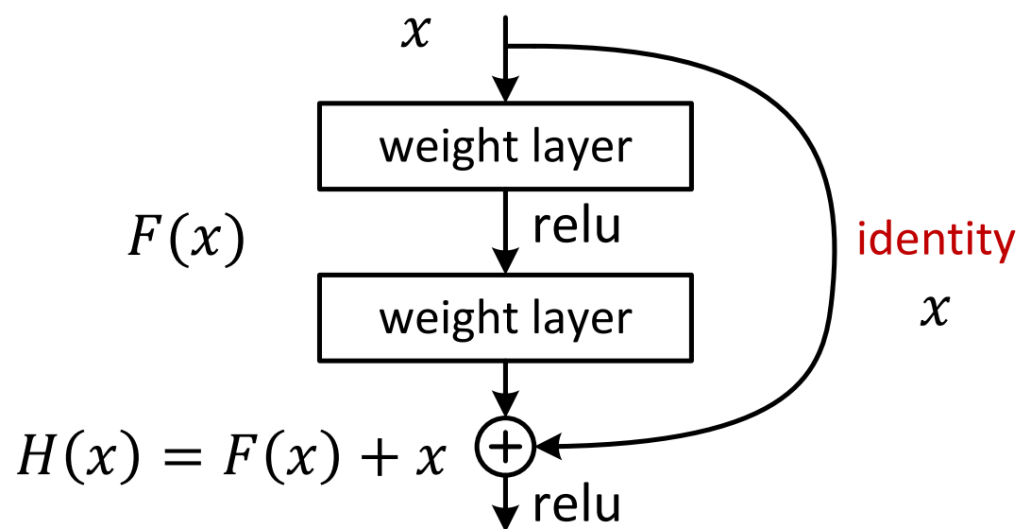
- Plain net



$H(x)$ is any desired mapping,
hope the small subnet fit $H(x)$

Deep Residual Learning

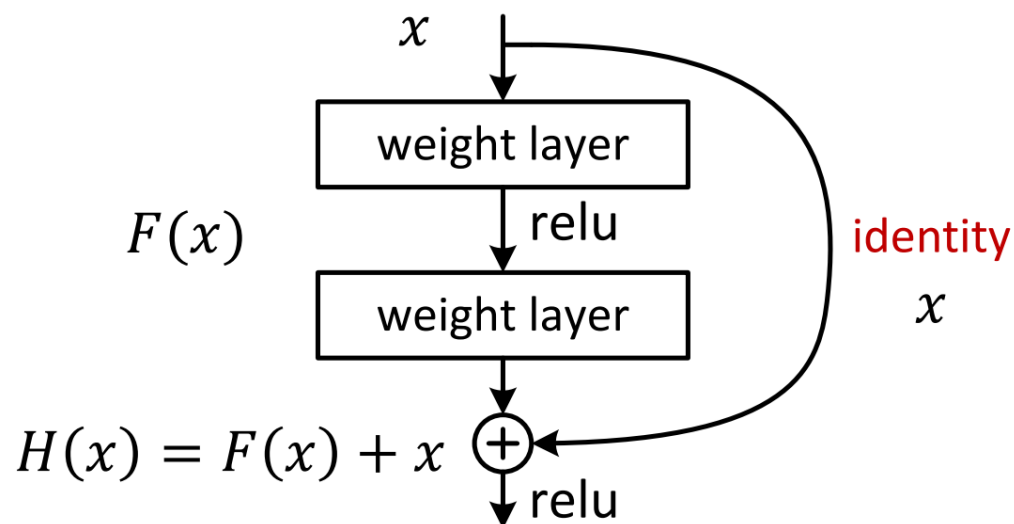
- Residual net



$H(x)$ is any desired mapping,
~~hope the small subnet fit $H(x)$~~
 hope the small subnet fit $F(x)$
 let $H(x) = F(x) + x$

Deep Residual Learning

- $F(x)$ is a **residual** mapping w.r.t. **identity**



- If identity were optimal, easy to set weights as 0
- If optimal mapping is closer to identity, easier to find small fluctuations

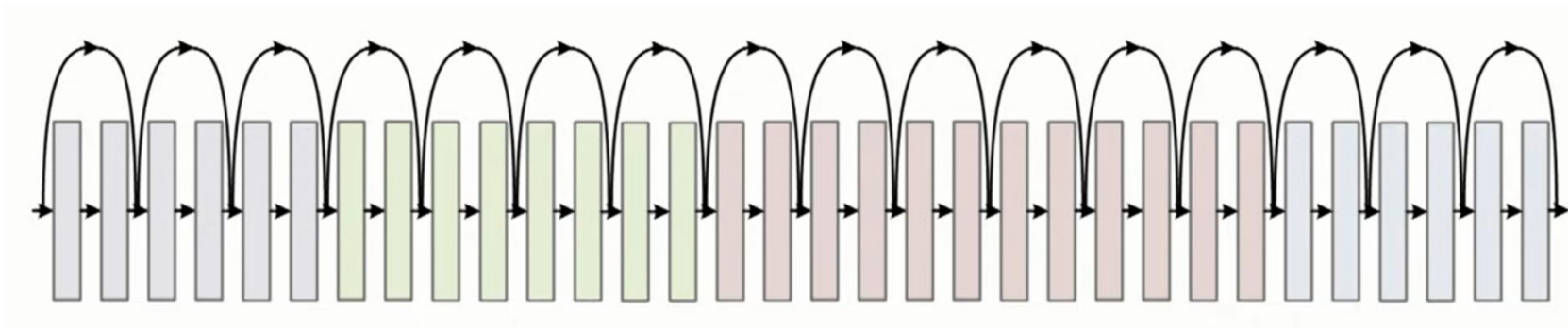
Deep Residual Learning

building residual nets

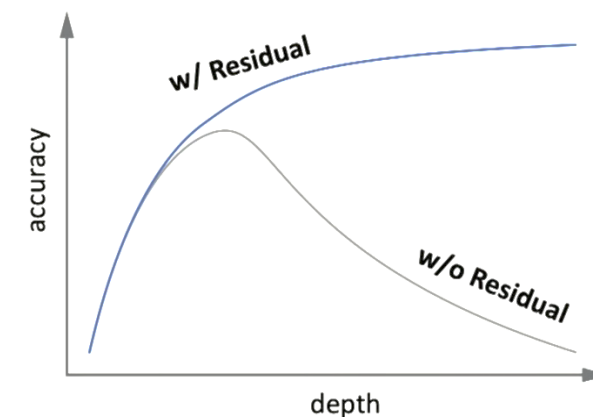
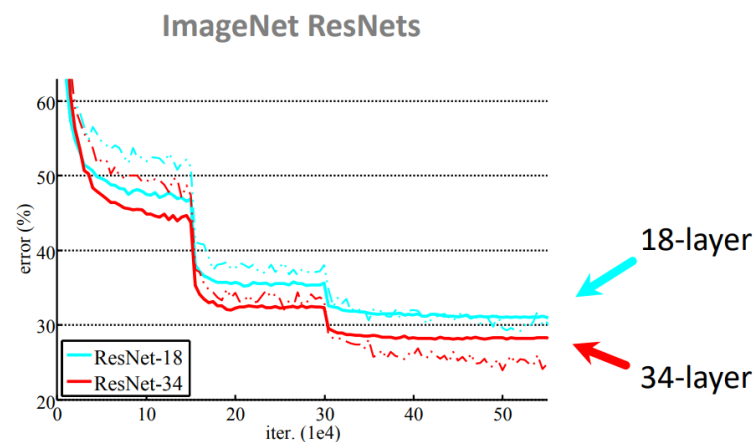
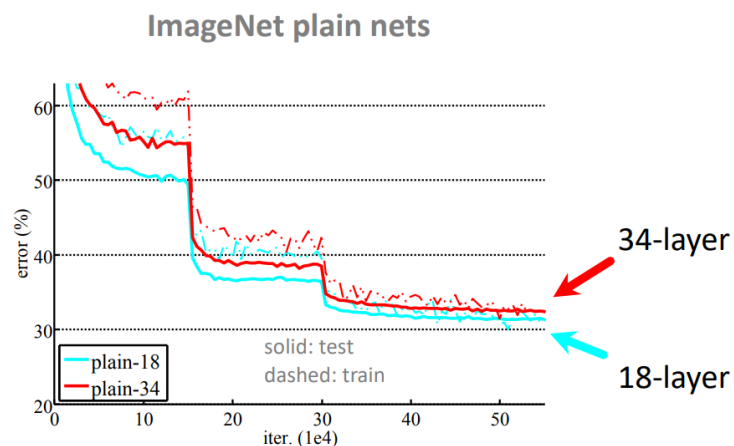
- add identity connections to vanilla nets

or equivalently:

- simply stack residual blocks
- blocks: the new generic modules for neural nets



Deep Residual Learning

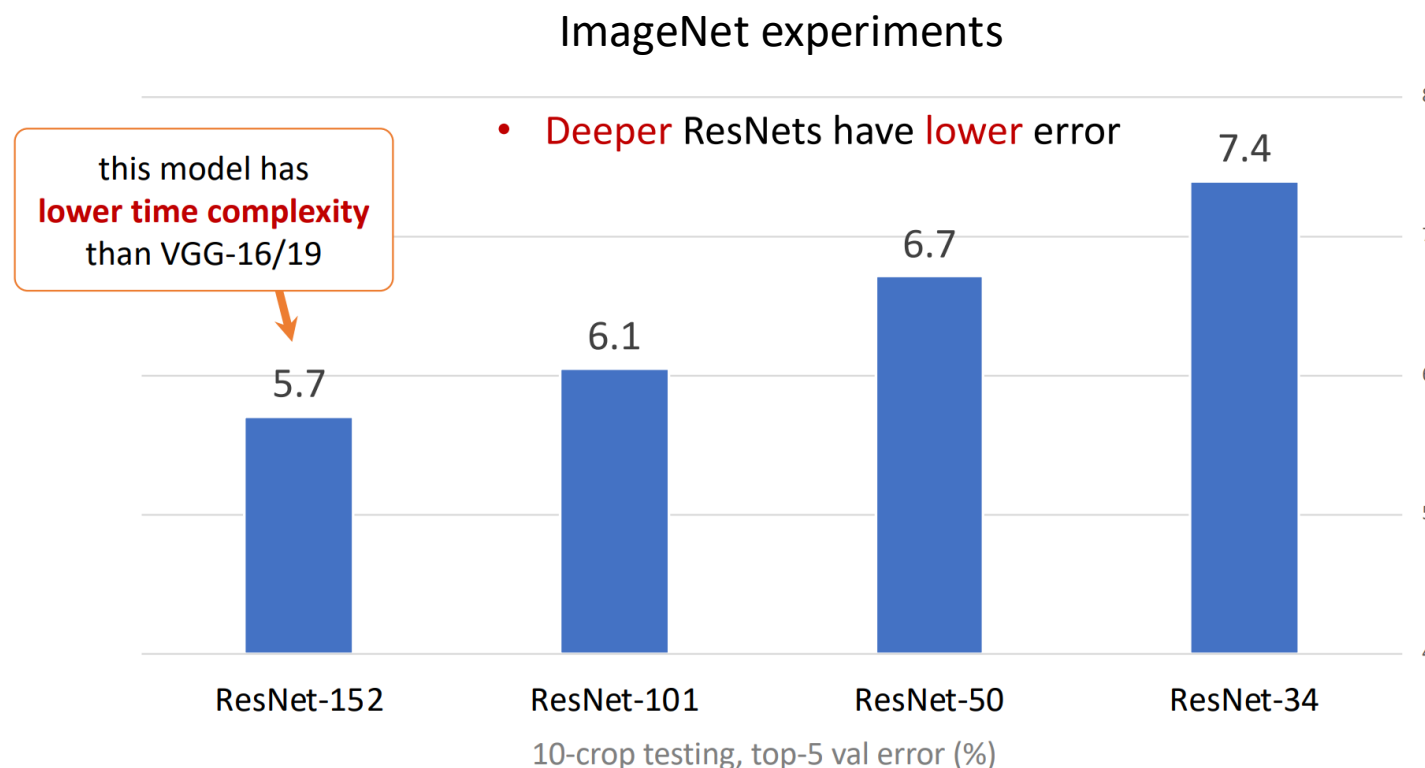


- Deep ResNets can be trained without difficulties
- Deeper ResNets have **lower training error**, and also lower test error

A checklist of training deep nets

All about signal propagation

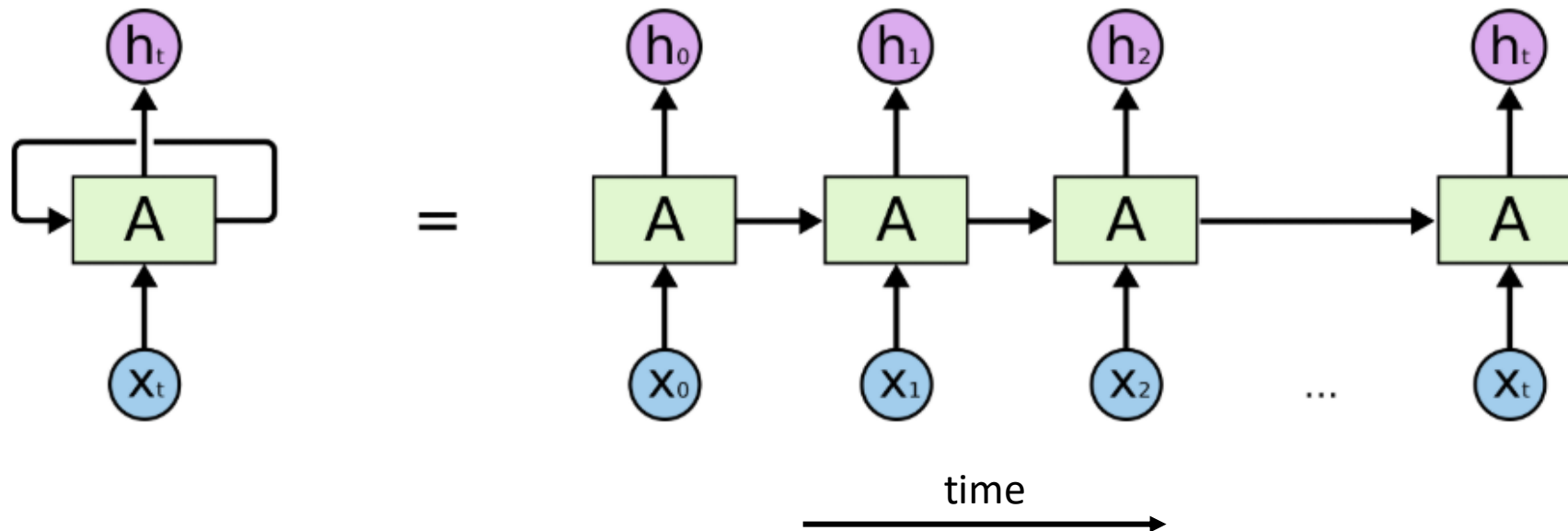
- ReLU (or similar)
- initialization
- normalization
- residual connection



Perception for Sequential Input

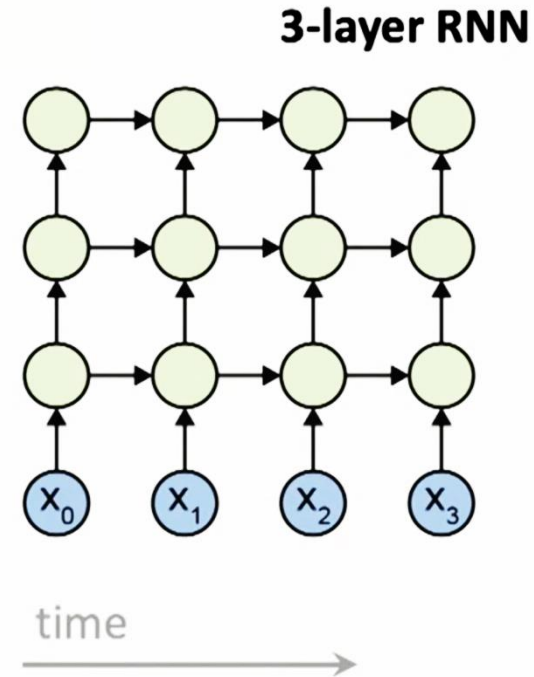
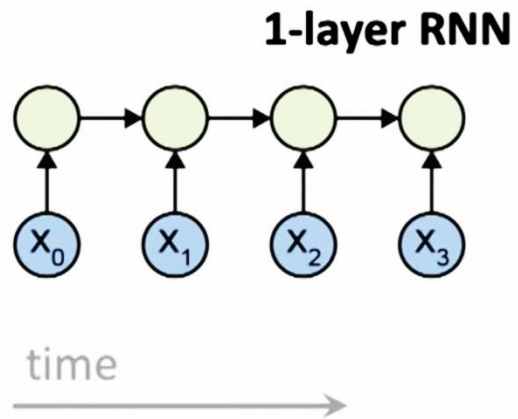
RNN for Sequence Modeling

- Recurrent Neural Networks (RNNs) have loops
- Unfold in time
 - temporally weight-sharing (vs. CNN, spatially)
 - temporally local connections



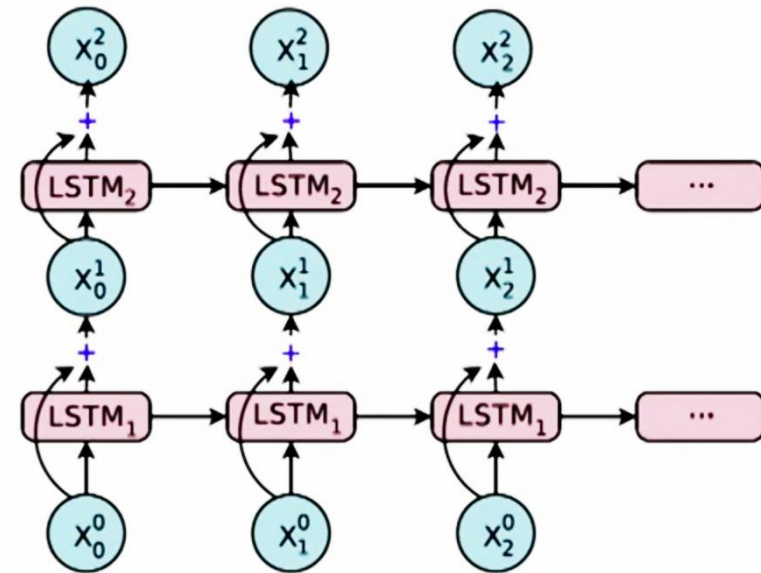
Going Deep with RNN

- view the states as a new sequence
- stack RNN units



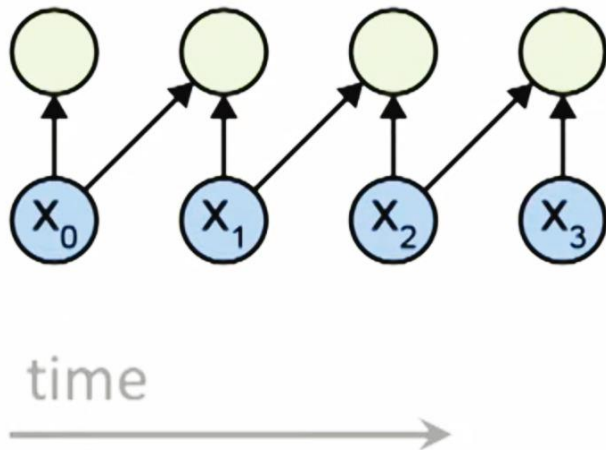
Example: Google's Neural MT System

- stack LSTM units (a form of RNN)
- going deep with residual connections
 - enable 16 layers
 - degrade after 4 layers if not using residual
 - like the observation of CNNs

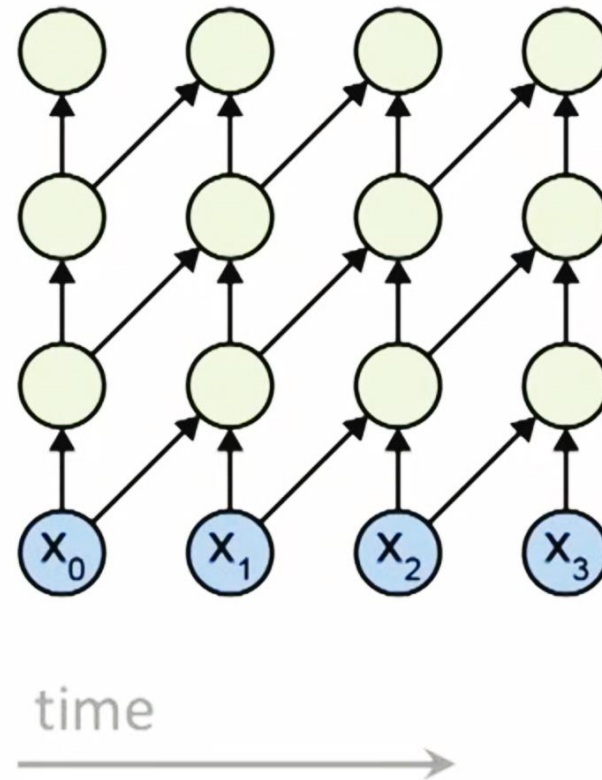


CNN for Sequence Modeling

- conv along 1-D sequence
- **causal** conv
 - Kernel doesn't see future



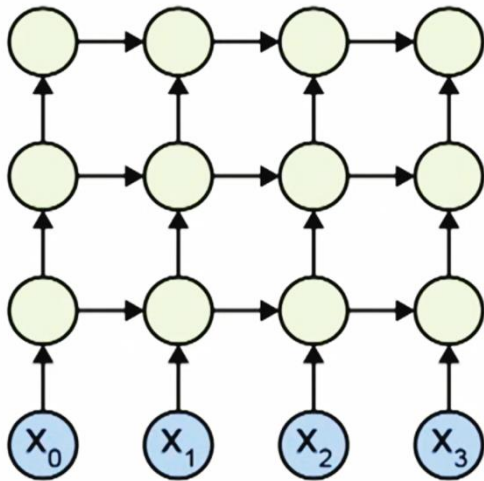
3-layer CNN



-
- The diagram illustrates a hierarchical generative model structure. It consists of four layers of nodes, represented by circles. The bottom layer (Layer 0) contains 16 blue nodes. The second layer (Layer 1) contains 12 white nodes. The third layer (Layer 2) contains 8 white nodes. The top layer (Layer 3) contains 1 orange node. Solid black arrows indicate the generative process, showing how nodes in lower layers generate nodes in higher layers. For example, two blue nodes in Layer 0 generate one white node in Layer 1, and so on, up to the single orange node in Layer 3. Dashed gray lines represent the receptive field or potential connections between nodes across layers.

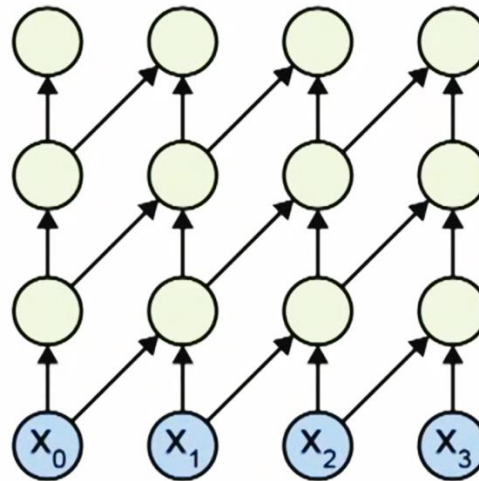


Sequence Modelling



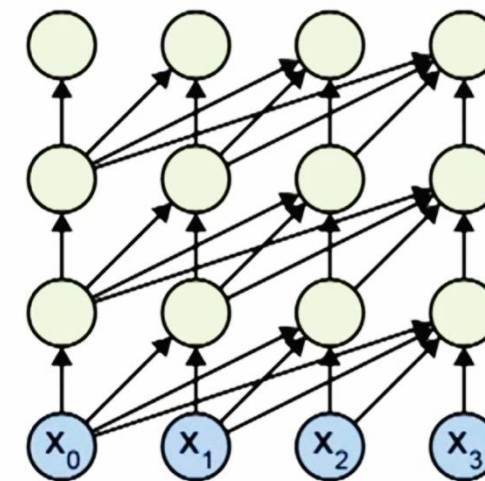
RNN

- full context
- not feedforward (parallel)



CNN

- limited context
- feedforward

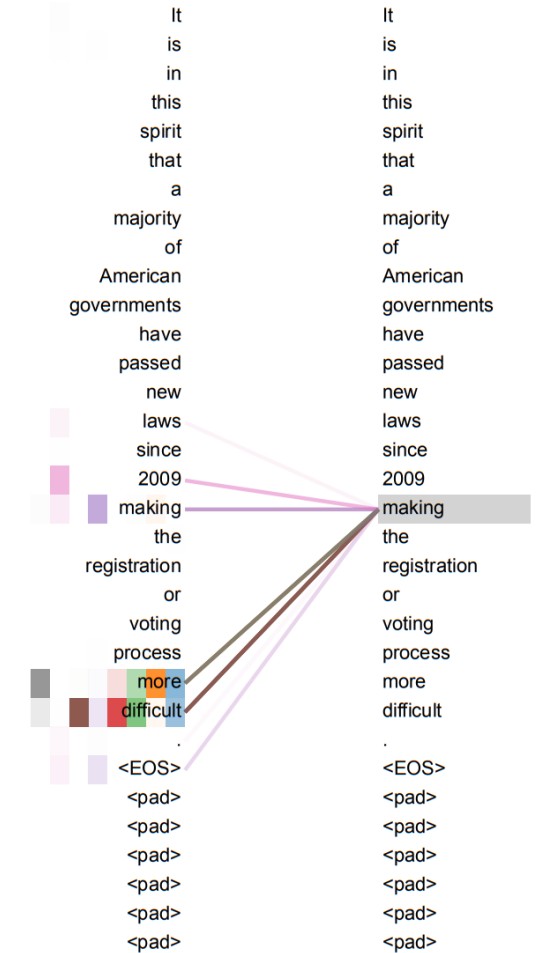
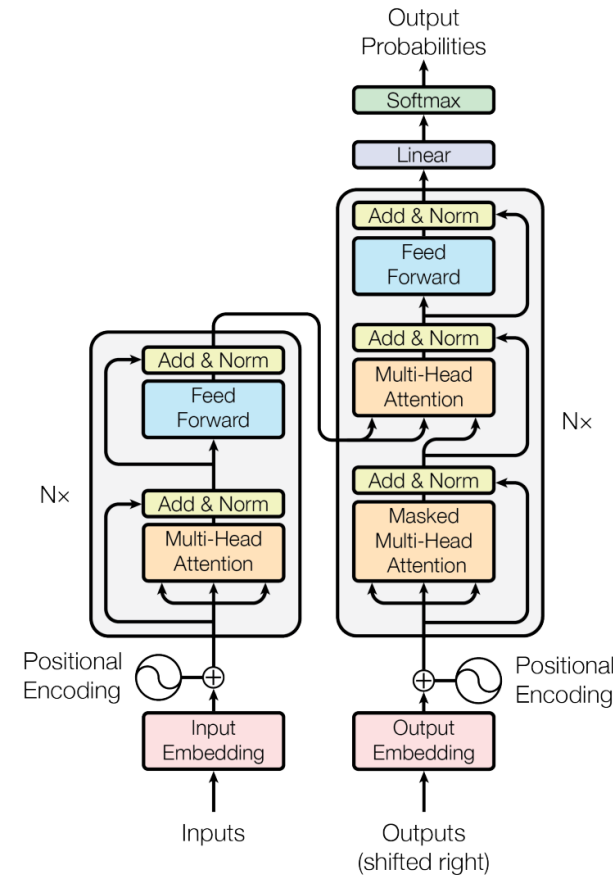


Attention

- every node can see previous all nodes
- full context
- feedforward

2017: Transformer

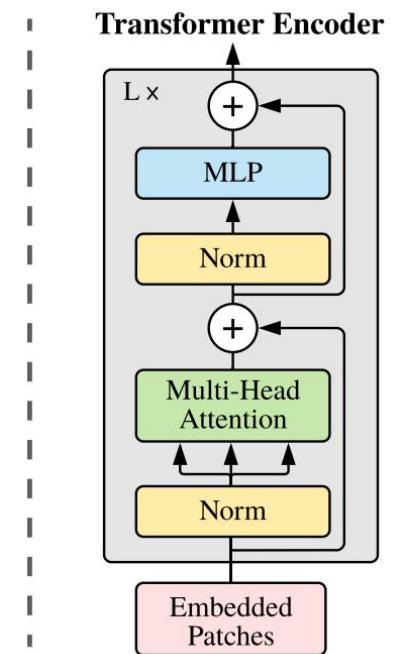
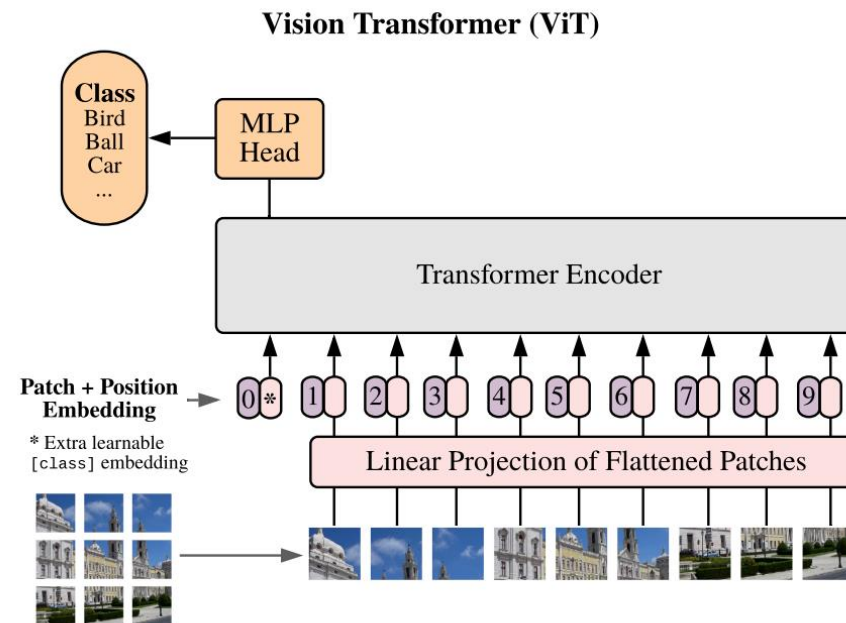
- every node can see previous all nodes
- attention is parameter-free
- Parameter layers are all feedforward
 - Q, K, V projections and MLP layers
 - all are 1x1 conv
- going deeper
 - LayerNorm + residual connection



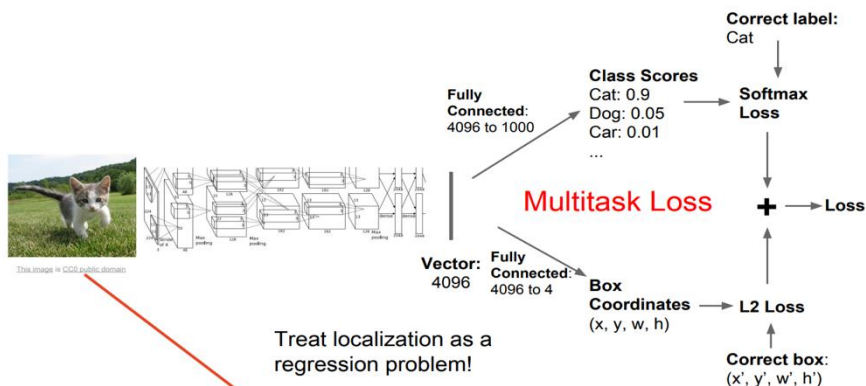
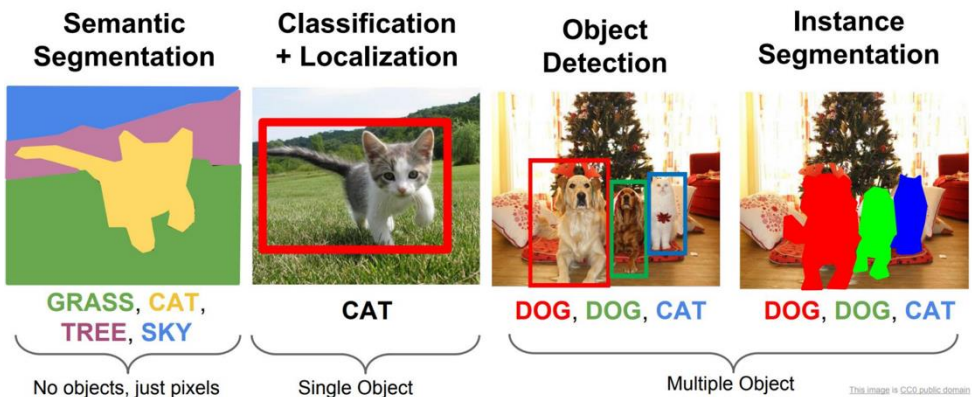
2020: Vision Transformer (ViT)

Back to Vision

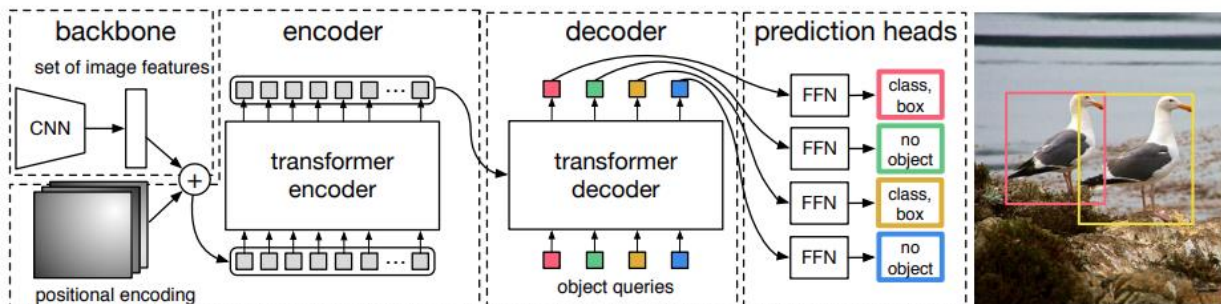
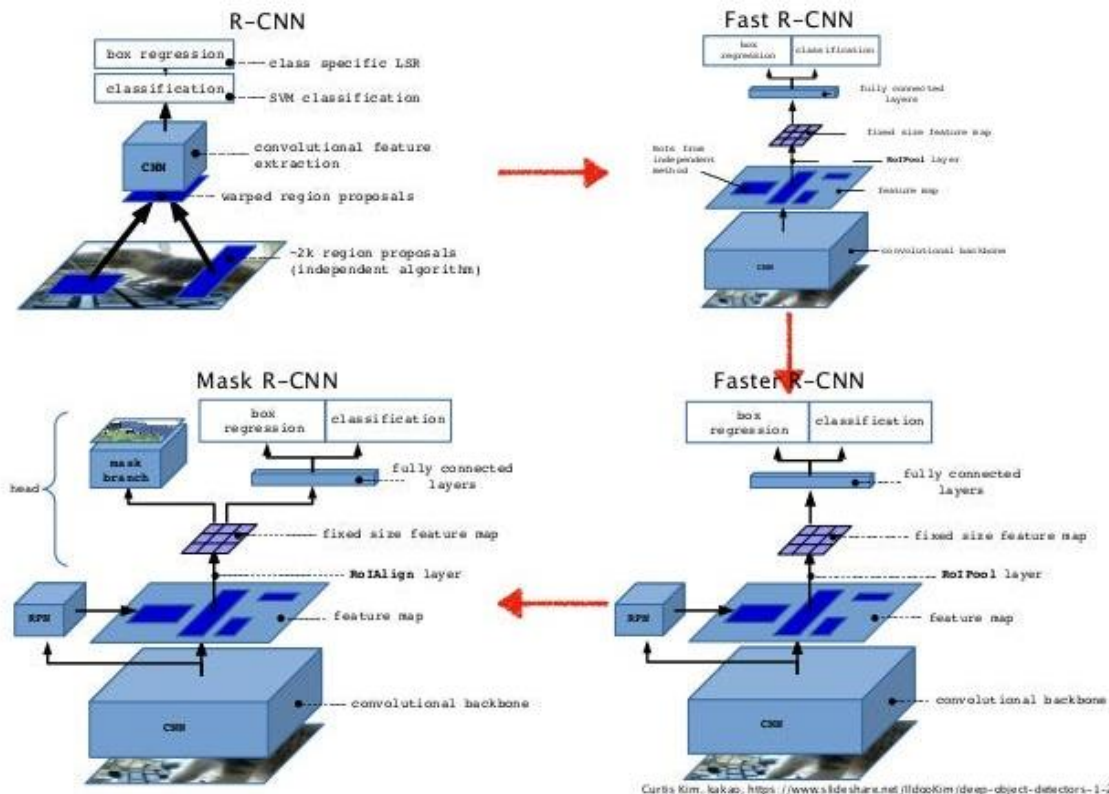
- sequences of image patches
- Transformers on patches
- unified architecture
 - language
 - vision
 - audio, 3d, sequences, ...



Applications based on NN



It is assumed that there is only one object in an input image.



- Girshick, Ross, et al. "Rich feature hierarchies for accurate object detection and semantic segmentation." CVPR 2014.
- Girshick, Ross. "Fast r-cnn." CVPR 2015.
- Ren, Shaoqing, et al. "Faster r-cnn: Towards real-time object detection with region proposal networks." NeurIPS 2015.
- He, Kaiming, et al. "Mask r-cnn." CVPR 2017.
- Carion, Nicolas, et al. "End-to-end object detection with transformers." ECCV 2020.

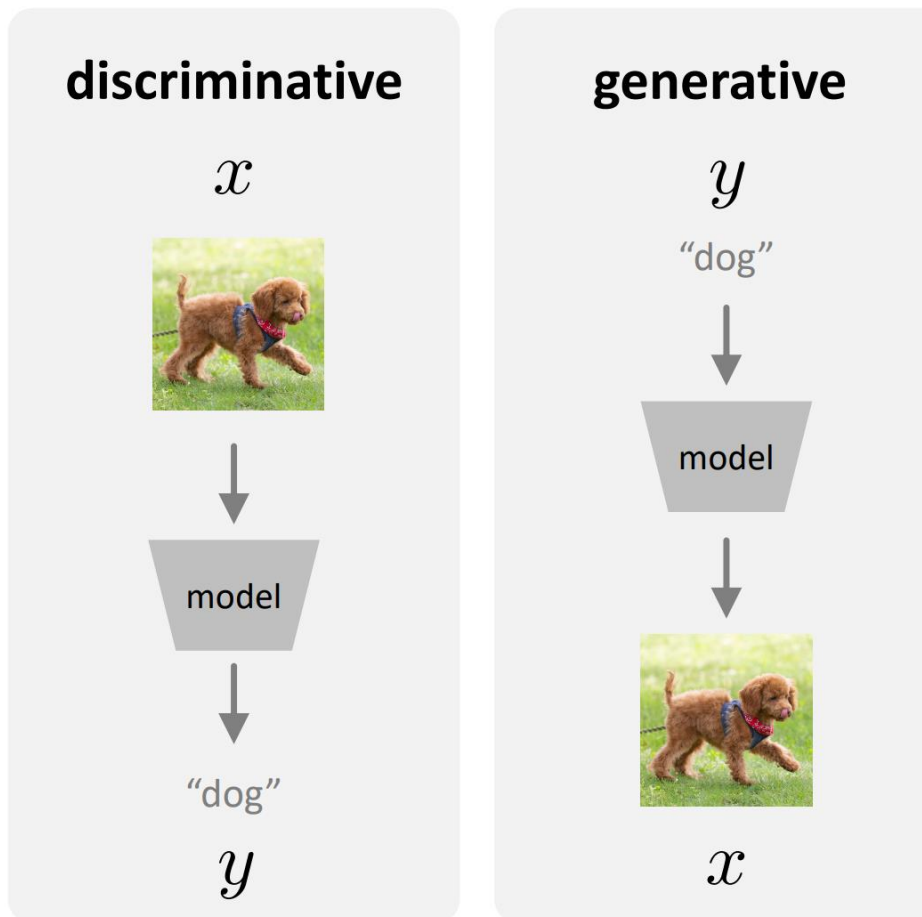
Perception (Discriminative) vs. Generative models

discriminative

- “sample” $x \Rightarrow$ “label” y
- one desired output

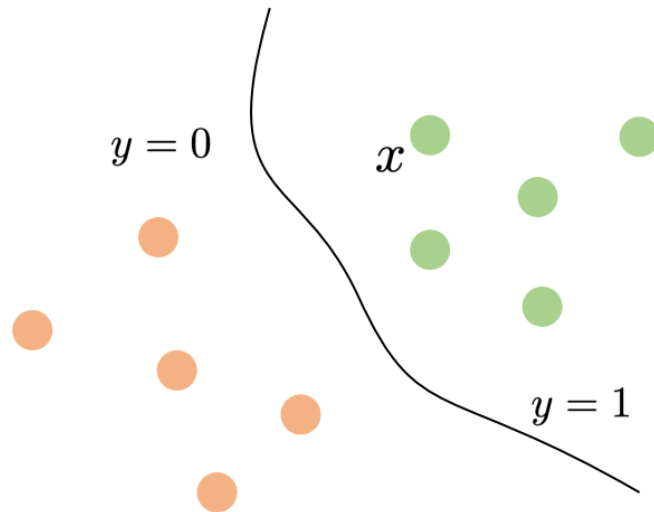
generative

- “label” $y \Rightarrow$ “sample” x
- many possible outputs

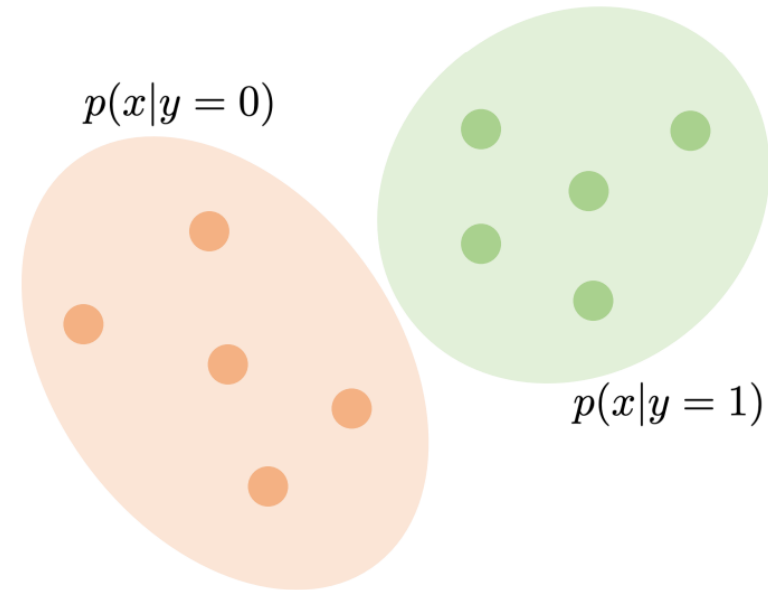


Discriminative vs. Generative models

discriminative $p(y|x)$



generative $p(x|y)$



What are Generative Models?

Gemini

2.5 Flash ▾

What are generative models?

◆ Generative models are a type of machine learning model designed to create new data that is similar to the data they were trained on. Unlike discriminative models, which learn to classify or predict outcomes based on given inputs (e.g., classifying an image as a "cat" or "dog"), generative models learn the underlying patterns and distribution of the data itself. This allows them to "generate" new, unique content, such as text, images, audio, and more. Think of it like this: a discriminative model can tell you if a picture is a cat, but a generative model can draw a new, original picture of a cat. [🔗](#) [🔗](#) [🔗](#) [🔗](#)

How Generative Models Work

Generative models learn to capture the joint probability distribution of a dataset. This means they learn how different features of the data relate to each other. For instance, in a model trained on images of human faces, it learns the relationships between the positions of eyes, the shape of a nose, and the presence of a mouth. After learning these complex relationships, the model can generate new data that follows the same statistical patterns. [🔗](#) [🔗](#) [🔗](#)

The training process for these models often involves unsupervised learning, where the model is given a large dataset without labels and must figure out the data's internal logic on its own. [🔗](#)

Ask Gemini

+ 🛠️ Tools



ChatGPT ▾

What are generative models?

Generative models are a class of machine learning models that learn the underlying distribution of data so they can generate new samples that resemble the training data.

In simple terms: instead of just learning to classify or predict (like *discriminative models*), generative models try to model how the data was created, allowing them to generate new, realistic-looking data points.

Key Ideas:

- They capture joint probability distributions $P(x, y)$ or just the data distribution $P(x)$.
- They can generate synthetic data, such as images, text, audio, or videos.
- They are often trained on large datasets and learn to mimic the patterns in them.

Examples of Generative Models:

1. **Gaussian Mixture Models (GMMs)** – early statistical generative models.
2. **Hidden Markov Models (HMMs)** – used for sequences like speech.
3. **Naive Bayes** (in a sense, since it models $P(x|y)$ and $P(y)$).
4. **Modern Deep Generative Models:**
 - **Variational Autoencoders (VAEs)** – generate data by sampling from a latent space.
 - **Generative Adversarial Networks (GANs)** – train a generator and discriminator in a minimax game.
 - **Diffusion Models** – iteratively denoise random noise into realistic samples (e.g., Stable Diffusion, DALL-E).

+ Ask anything

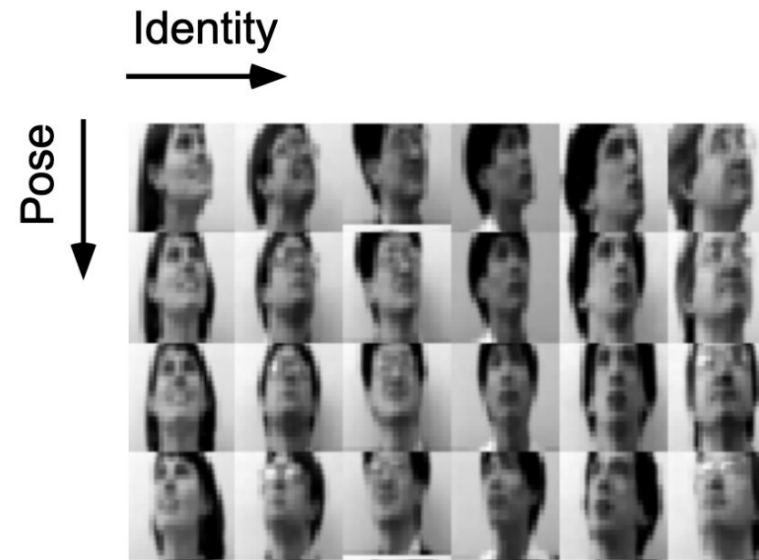


Probabilistic modeling

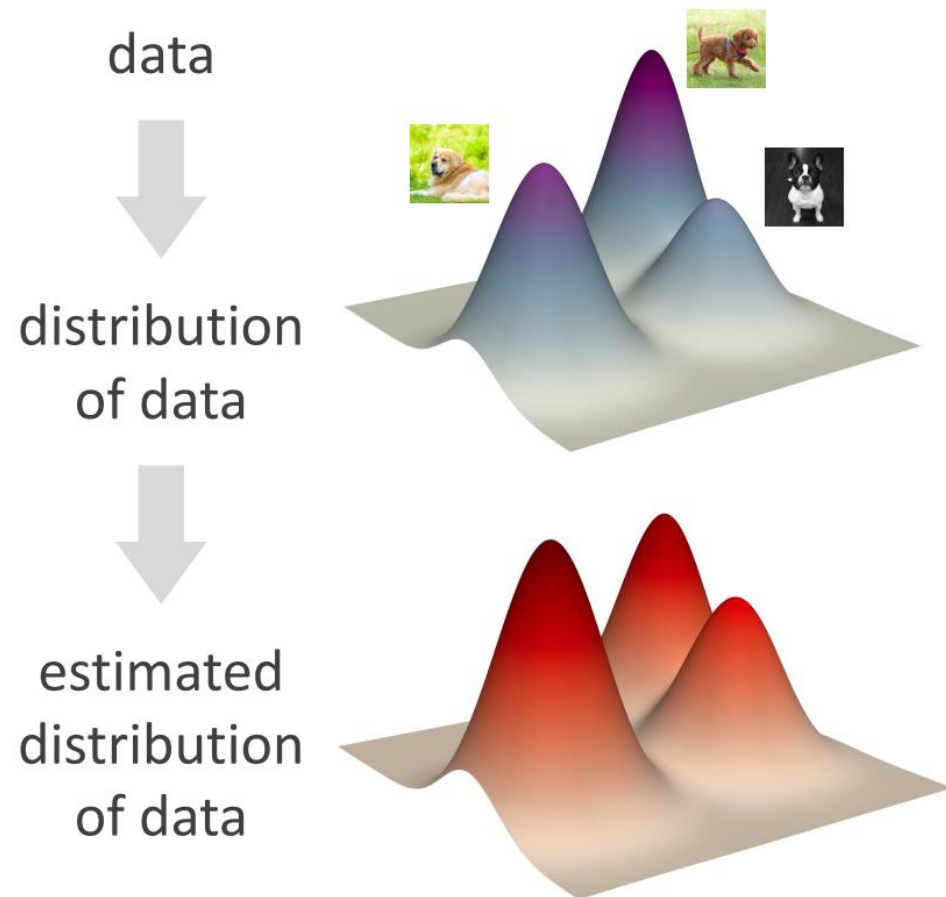
- Where does probability come from?
- Assuming underlying **distributions of data generation process**

example:

- latent factors z (pose, lighting, scale, ...)
 - z has simple distributions
 - observations x are rendered by a “world model” that’s a function on z
 - observations x have complex distributions
-
- Probability is part of the modeling.



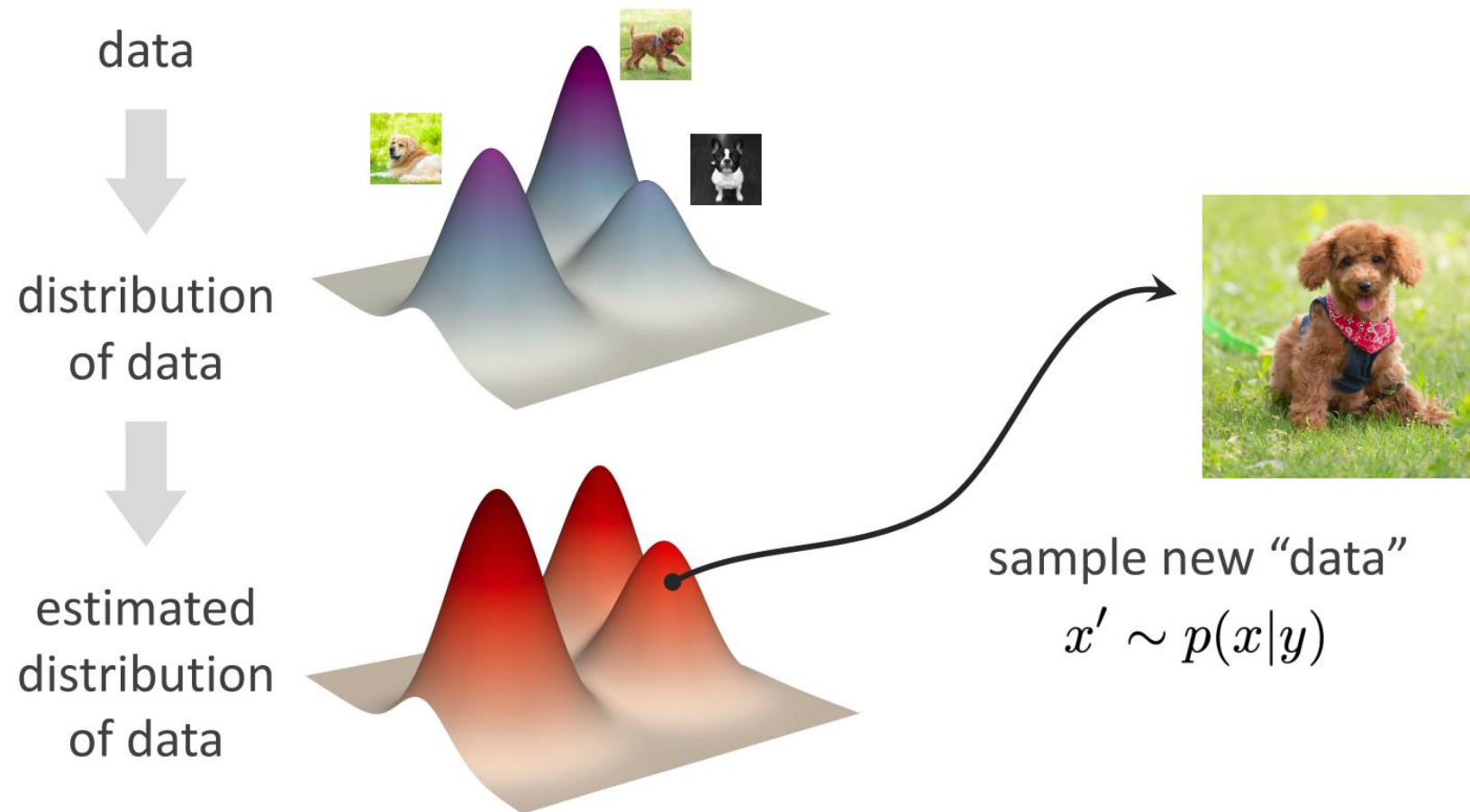
Probabilistic modeling



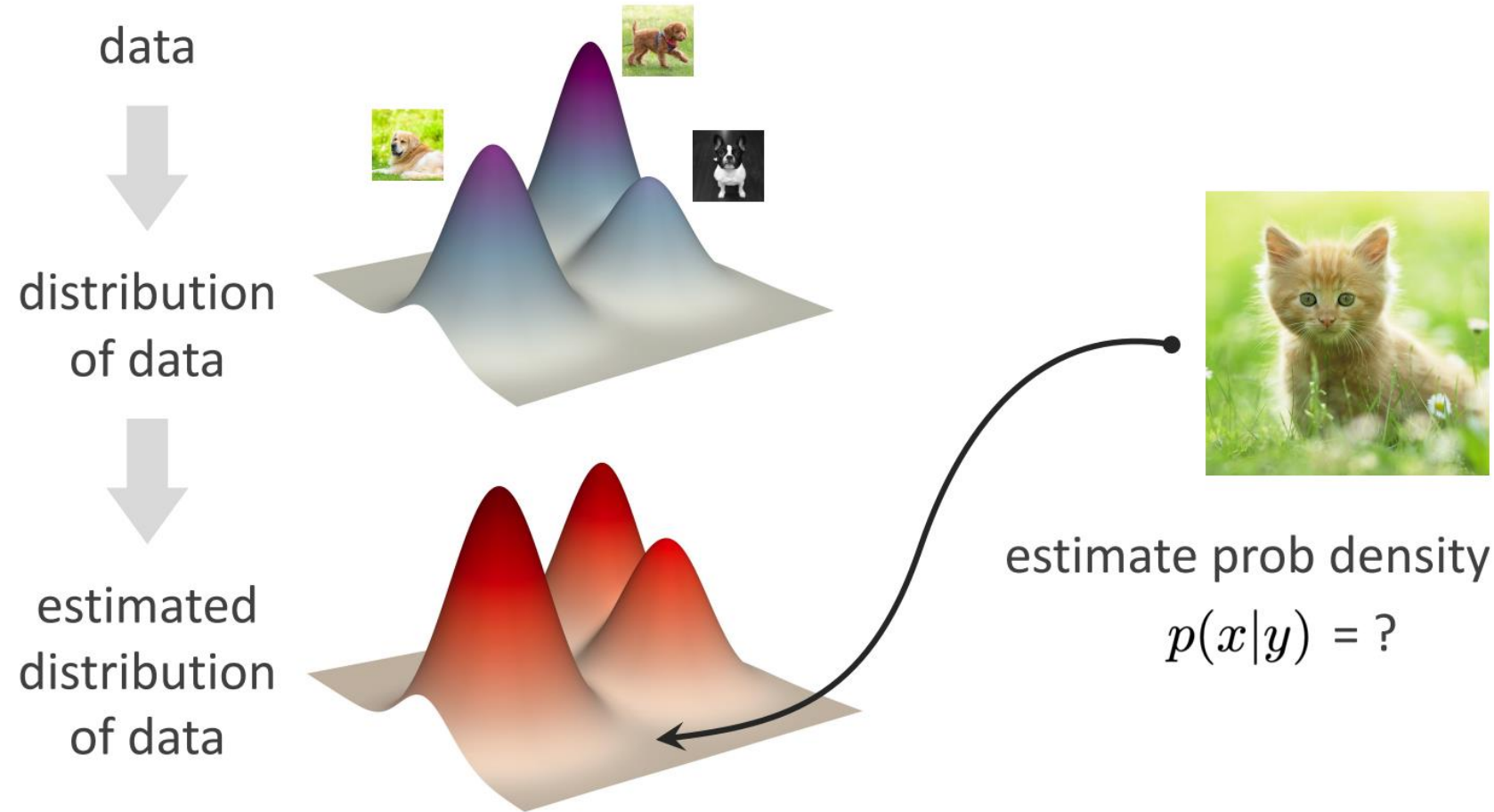
- Optimize a loss function

$$\mathcal{L}(\text{distribution of data}, \text{estimated distribution of data})$$

Probabilistic modeling



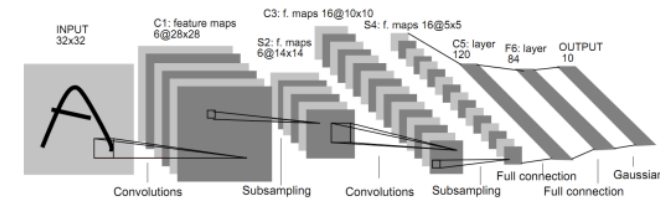
Probabilistic modeling



Generative Models

- Deep learning is **representation learning**
- Learning to represent data instances
 - map data to feature: $x \rightarrow f(x)$
 - minimize loss w/ target: $\mathcal{L}(y, f(x))$

$$x \xrightarrow{\quad} f(x)$$



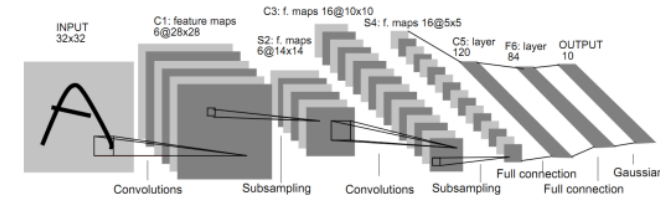
Generative Models

- Deep learning is **representation learning**

$$x \longrightarrow f(x)$$

- Learning to represent data instances

- map data to feature: $x \rightarrow f(x)$
- minimize loss w/ target: $\mathcal{L}(y, f(x))$



$$g(\pi) \longleftarrow \pi$$

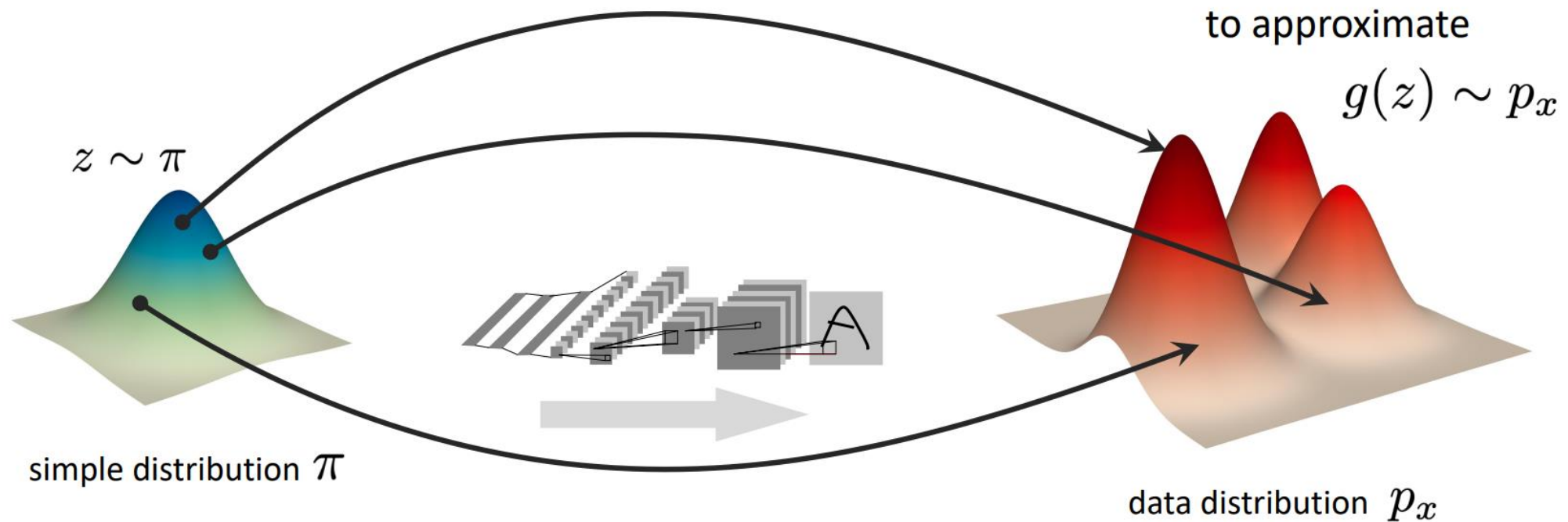
- Learning to **represent probability distributions**

- map a simple distribution (Gaussian/uniform) to a complex one: $\pi \rightarrow g(\pi)$
- minimize loss w/ data distribution: $\mathcal{L}(p_x, g(\pi))$

- Often perform both together

Learning to represent probability distributions

- From simple to complex distributions



Generative Models may involve:

- **Formulation:**
 - formulate a problem as probabilistic modeling
 - decompose complex distributions into simple and tractable ones
- **Representation:**
 - deep neural networks to represent data and their distributions
- **Objective function:**
 - to measure how good the predicted distribution is
- **Optimization:**
 - optimize the networks and/or the decomposition
- **Inference:**
 - sampler: to produce new samples
 - probability density estimator (optional)

Takeaways

Perception Model

- Representation Learning
 - from raw data to abstraction
 - a common problem across areas
- Going deep with neural nets
 - compose simple modules to learn complex functions
 - general tools across different areas

Generative Model

- Learning to represent probability distributions
 - map a simple distribution (Gaussian/uniform) to a complex one: $\pi \rightarrow g(\pi)$
 - minimize loss w/ data distribution: $\mathcal{L}(p_x, g(\pi))$
- Often perform both together



Thank You!